MASTER 2 INTELLIGENCE ARTIFICIELLE

Traitement Automatique du Langage Naturel

TP 1 — Introduction au NLP

Prétraitement, Représentation et Analyse de Textes

Corpus : Le Comte de Monte-Cristo — Alexandre Dumas (1844)

🎓 **Niveau**	Master 2 Intelligence Artificielle
⏱️ **Durée**	4 heures
🐍 **Langage**	Python 3.9+
📦 **Librairies**	NLTK · SpaCy · Scikit-Learn · Pandas · Matplotlib · WordCloud
📖 **Corpus**	Le Comte de Monte-Cristo (Project Gutenberg — domaine public)
🔗 **Source**	https://www.gutenberg.org/files/17989/17989-0.txt

Rendu — code source complet et exécuté : https://github.com/sephorataty/TP1-NLP-Monte-Cristo (notebook `TP1_NLP_Monte_Cristo.ipynb`, figures, données et ce rapport). Tous les chiffres ci-dessous proviennent de l'exécution réelle du notebook sur le Tome I du roman (Gutenberg #17989), avec Python 3.13, NLTK 3.10, SpaCy 3.8 (fr_core_news_sm) et scikit-learn 1.9.

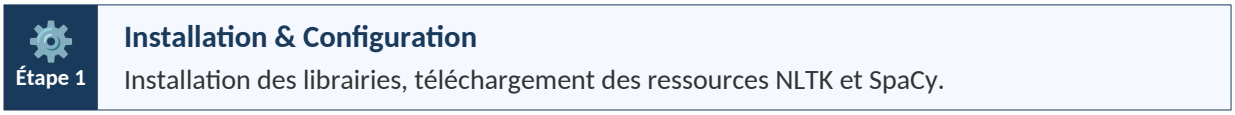
🎯 Objectifs pédagogiques

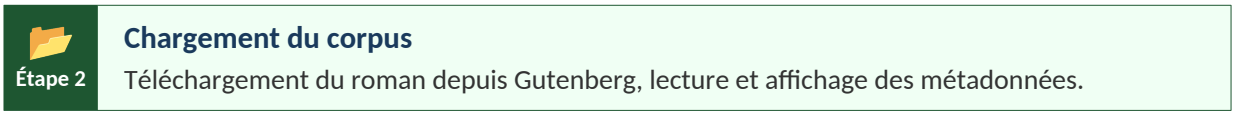
À l'issue de ce TP, vous serez capable de :

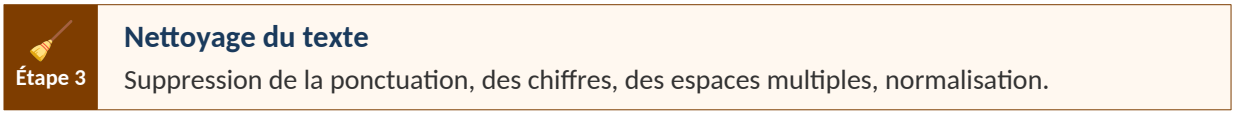
- Comprendre et mettre en œuvre un pipeline NLP complet de A à Z
- Charger, nettoyer et normaliser un corpus textuel brut
- Réaliser la tokenisation avec NLTK et SpaCy
- Effectuer une analyse fréquentielle des mots
- Supprimer les stop words et mesurer leur impact
- Visualiser les mots importants via des WordClouds
- Appliquer le stemming (racinisation) et la lemmatisation
- Construire une matrice Document-Terme
- Représenter un corpus avec Bag of Words (BoW)
- Représenter un corpus avec TF-IDF
- Comparer BoW et TF-IDF et analyser leurs différences

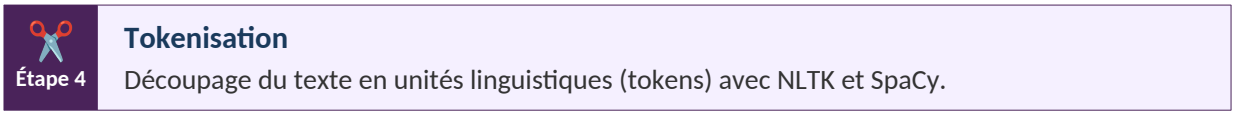
Pipeline NLP — Vue d'ensemble

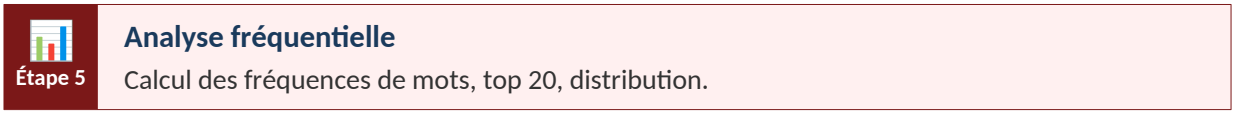
Avant de commencer, voici les 11 étapes que vous allez réaliser dans ce TP. Chaque étape s'appuie sur la précédente — il est important de les réaliser dans l'ordre.

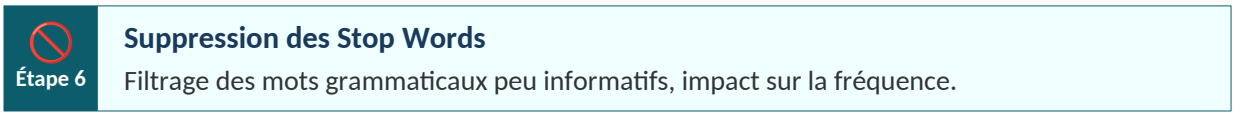


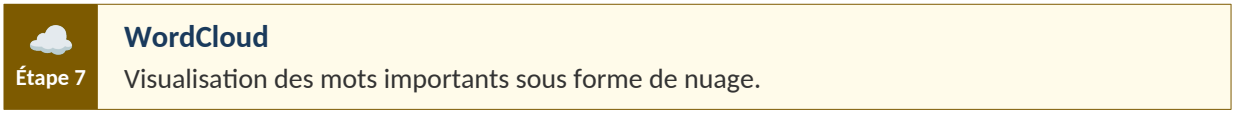




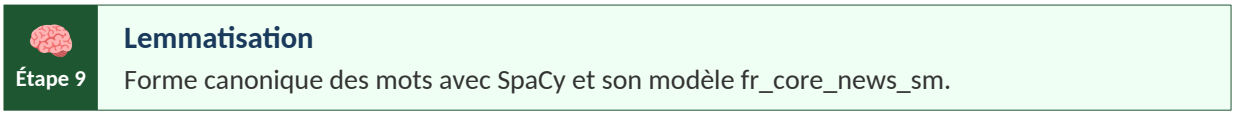


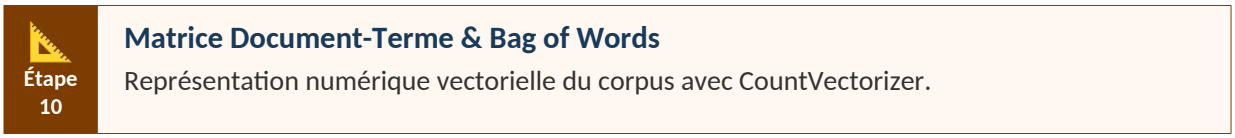


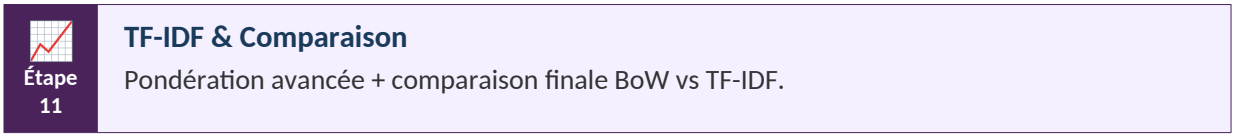












Partie 1

Installation et Configuration

💡 **Pourquoi ces librairies ?**

NLTK : librairie historique du NLP en Python (tokenisation, stop words, stemming, lemmatisation)
SpaCy : librairie moderne, plus rapide, idéale pour le français et les pipelines industriels
Scikit-Learn : modèles ML + vectorisation BoW et TF-IDF (CountVectorizer, TfidfVectorizer)
Pandas : manipulation de données tabulaires (DataFrames)
Matplotlib : visualisations graphiques
WordCloud : génération de nuages de mots

1.1 Installation des librairies

```
# Dans votre terminal (une seule fois)
pip install nltk spacy scikit-learn pandas matplotlib wordcloud requests
seaborn
```

1.2 Imports principaux

```
import requests
import re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from wordcloud import WordCloud

import nltk
import spacy
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer
from nltk.stem import WordNetLemmatizer

from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

1.3 Téléchargement des ressources NLTK

```
import nltk

nltk.download('punkt')       # Tokeniseur de phrases et mots
nltk.download('punkt_tab')   # Ressource complémentaire (versions récentes
de NLTK)
nltk.download('stopwords')   # Listes de mots vides (français, anglais...)
nltk.download('wordnet')     # Base lexicale pour la lemmatisation
```

```
nltk.download('omw-1.4')    # Open Multilingual Wordnet (multilingue)
```

> ⚠️ **Erreur fréquente : LookupError punkt_tab not found**
>
> Si vous obtenez cette erreur lors de l'utilisation de word_tokenize :
>
> LookupError: Resource punkt_tab not found
>
> Solution : exécutez nltk.download('punkt_tab')
>
> Ce téléchargement n'est nécessaire qu'une seule fois par machine.

1.4 Téléchargement du modèle SpaCy

```
# Dans votre terminal :
python -m spacy download fr_core_news_sm

# Vérification dans Python :
import spacy
nlp = spacy.load('fr_core_news_sm')
print('Modèle SpaCy chargé avec succès !')
```

> ✏️ **Travail demandé**
>
> a) Exécutez toutes les installations et vérifiez qu'elles fonctionnent.
>
> b) Affichez la version de chaque librairie installée :
>
> import nltk, spacy, sklearn, pandas
>
> print(nltk.__version__, spacy.__version__, sklearn.__version__, pandas.__version__)
>
> c) Vérifiez que le modèle SpaCy est chargé en analysant la phrase :
>
> doc = nlp('Edmond Dantès fut injustement emprisonné au château d\'If.')
>
> print([(t.text, t.pos_) for t in doc])

Réponse :

a) Installations

Toutes les librairies s'installent et s'importent sans erreur (`pip install -r requirements.txt` puis `python -m spacy download fr_core_news_sm`). Les 5 ressources NLTK (punkt, punkt_tab, stopwords, wordnet, omw-1.4) sont téléchargées ; `punkt_tab` est bien indispensable avec NLTK ≥ 3.9, sinon `word_tokenize` lève l'erreur `LookupError` décrite dans le sujet.

b) Versions installées

Python	NLTK	SpaCy	scikit-learn	pandas	Matplotlib	WordCloud	Modèle SpaCy
3.13	3.10.4	3.8.18	1.9.1	3.0.5	3.11.2	1.9.6	fr_core_news_sm 3.8.0

c) Vérification du modèle SpaCy

```
[('Edmond', 'PROPN'), ('Dantès', 'PROPN'), ('fut', 'AUX'), ('injustement', 'ADV'),
 ('emprisonné', 'VERB'), ('au', 'ADP'), ('château', 'NOUN'), ("d'", 'ADP'), ('If',
'PROPN'), ('.', 'PUNCT')]
```

Le modèle est opérationnel (pipeline : tok2vec, morphologizer, parser, attribute_ruler, lemmatizer, ner). L'étiquetage est entièrement correct : les trois noms propres sont reconnus, « fut » est bien un auxiliaire, « emprisonné » un participe verbal, et l'élision « d' » est séparée de « If » et étiquetée préposition.

Partie 2

Chargement du corpus

Le Comte de Monte-Cristo d'Alexandre Dumas est disponible gratuitement sur le **Project Gutenberg**, une bibliothèque numérique de livres du domaine public. Nous allons le télécharger directement depuis l'URL officielle.

2.1 Téléchargement du roman

```
import requests

URL = 'https://www.gutenberg.org/files/17989/17989-0.txt'

# Téléchargement
response = requests.get(URL)
response.encoding = 'utf-8'
raw_text = response.text

print(f'Téléchargement terminé.')
print(f'Taille brute : {len(raw_text):,} caractères')
print(f'Aperçu :\n{raw_text[:300]}')
```

2.2 Extraction du contenu (sans les métadonnées Gutenberg)

```
# Le fichier Gutenberg contient un en-tête et un pied de page
# On extrait uniquement le roman
DEBUT = '*** START OF THE PROJECT GUTENBERG EBOOK 17989 ***'
FIN   = '*** END OF THE PROJECT GUTENBERG EBOOK 17989 ***'

roman = raw_text.split(DEBUT)[1].split(FIN)[0]

print(f'Taille du roman (sans métadonnées) : {len(roman):,} caractères')
print(f'Nombre de lignes : {roman.count(chr(10)):,}')
print(f'Aperçu :\n{roman[:500]}')
```

2.3 Sauvegarde locale (optionnel)

```
# Sauvegarder le roman en local pour éviter de re-télécharger
with open('monte_cristo.txt', 'w', encoding='utf-8') as f:
    f.write(roman)

# Chargement depuis le fichier local :
# with open('monte_cristo.txt', encoding='utf-8') as f:
#     roman = f.read()
```

✏️ **Travail demandé**

a) Téléchargez le roman et affichez :

- Le nombre total de caractères (len)
- Le nombre total de lignes (count('\n'))
- Les 500 premiers caractères
- Les 200 derniers caractères (pour voir la fin du fichier Gutenberg)

b) Comptez le nombre de mots en utilisant simplement .split()
(estimation rapide, avant toute tokenisation avancée)

c) Combien de fois apparaît le nom 'Dantès' dans le roman ?
Utilisez roman.lower().count('dantès')

d) Cherchez les 5 premières lignes contenant le mot 'vengeance'.

e) Estimez le nombre de pages du roman (supposer 300 mots / page).

Réponse :

Remarque préalable : corpus et marqueurs Gutenberg

Le fichier 17989 correspond au **Tome I** du roman (sur 4), soit les chapitres I à XXXI (de l'arrivée du Pharaon à Marseille jusqu'à « Simbad le marin »). Par ailleurs, les marqueurs indiqués dans le sujet (`*** START OF THE PROJECT GUTENBERG EBOOK 17989 ***`) n'existent pas dans le fichier : le vrai marqueur est `*** START OF THIS PROJECT GUTENBERG EBOOK LE COMTE DE MONTE-CRISTO, TOME I ***`. Le code d'origine lève donc une `IndexError` sur `split(DEBUT)[1]`. Nous utilisons une expression régulière robuste aux deux formats (THE/THIS, avec ou sans titre), et un cache local + un miroir GitHub du Project Gutenberg si le site est inaccessible.

a) Statistiques de base

Mesure	Valeur
Caractères (fichier brut)	747 000
Caractères (roman extrait)	**727 472**
Lignes (roman extrait)	**16 686**
Métadonnées supprimées	19 528 caractères (licence en anglais)

Les **500 premiers caractères** contiennent la page de titre (« Alexandre Dumas — LE COMTE DE MONTE-CRISTO — Tome I (1845-1846) ») puis la table des matières (« I Marseille.--L'arrivée. II Le père et le fils. III Les Catalans… »). Les **200 derniers caractères du fichier brut** sont la fin de la licence Gutenberg (« … subscribe to our email newsletter… * **END: FULL LICENSE** * ») : sans extraction, ce texte anglais serait analysé. Après extraction, le roman se termine bien par « …sous les enchantements de ce rêve inouï. FIN DU TOME PREMIER ».

b) à e)

- **b)** `len(roman.split())` = **121 578 mots** (estimation brute).
- **c)** `roman.lower().count('dantès')` = **777 occurrences**.
- **d)** 12 lignes contiennent « vengeance ». Les 5 premières : « que Fernand surtout était terrible dans sa vengeance.» » ; « «À la bonne heure, continua Danglars; ainsi votre vengeance aurait le » ; « menaçant et fort pour toutes les vengeances; alors il manifesta à M. » ; « qui, pour lui aussi, était devenu messager d'une rude vengeance. Alors, » ; « Il se disait que c'était la haine des hommes et non la vengeance de Dieu ».
- **e)** 121 578 / 300 ≈ **405 pages** pour le Tome I (≈ 1 600 pages pour l'œuvre complète en 4 tomes).

Le découpage en chapitres (regex sur les numéros romains isolés) détecte bien les **31 chapitres**, de 1 866 mots (ch. XX) à 9 363 mots (ch. XXXI).

❓ Questions de réflexion

Q1. Pourquoi est-il important d'extraire uniquement le contenu du roman et de supprimer les métadonnées Gutenberg avant toute analyse ?

Q2. Quelle est la différence entre compter les mots avec .split() et utiliser word_tokenize ? Quand préférer l'un ou l'autre ?

Q3. La fréquence d'un nom de personnage est-elle un bon indicateur de son importance dans le

roman ? Justifiez.

Réponse :

Q1. Pourquoi supprimer les métadonnées Gutenberg ?

L'en-tête et surtout la licence finale représentent ≈ 19 500 caractères **en anglais** qui n'appartiennent pas à l'œuvre. Les conserver : (1) ajoute au vocabulaire des mots parasites (project, gutenberg, ebook, license, foundation…) ; (2) fausse toutes les statistiques (nombre de mots, fréquences, richesse lexicale, hapax) ; (3) pollue les représentations BoW/TF-IDF du dernier bloc et la détection de langue ; (4) rend les résultats non comparables d'une édition à l'autre. Le principe général : **analyser uniquement le signal**, jamais l'emballage.

Q2. `.split()` vs `word_tokenize`

`.split()` coupe seulement sur les espaces : la ponctuation reste collée aux mots, si bien que « Dantès! », « Dantès, », « «Dantès » deviennent des types différents. Sur le texte brut, `.split()` donne 121 578 mots mais **20 220 types**, contre **11 142 types** après nettoyage + `word_tokenize`. `word_tokenize` applique des règles linguistiques (séparation de la ponctuation, abréviations comme « M. », contractions, nombres). **On préfère `.split()`** pour une estimation rapide ou sur un texte déjà nettoyé (c'est ce que fait la partie 10), et **`word_tokenize`** dès que l'on fait de l'analyse (fréquences, vocabulaire, vectorisation).

Q3. La fréquence d'un nom est-elle un indicateur d'importance ?

Seulement partiellement. Dantès est bien le plus fréquent (777 + 247 « Edmond »), mais : (1) les personnages sont souvent désignés autrement (« le jeune homme » : 140 bigrammes, « le prisonnier », « le numéro 34 », pronoms) ; (2) dans ce tome, Dantès change d'identité (abbé Busoni, Simbad le marin) : au-delà du chapitre XXV son nom disparaît presque alors qu'il est toujours au centre du récit (voir partie 10) ; (3) l'abbé Faria, pivot du roman, n'apparaît que **105 fois** sous son nom mais 214 fois comme « l'abbé ». L'importance narrative relève du rôle (qui agit, sur qui), ce que mesurent mieux la résolution de coréférence, la NER ou l'analyse de réseaux de personnages.

Partie 3

Nettoyage du texte

Un texte brut est rarement exploitable directement. Il contient de nombreux éléments parasites : ponctuation, chiffres, majuscules, retours à la ligne multiples, caractères spéciaux typographiques du XIXe siècle (tirets cadratins, guillemets français, etc.).

3.1 Identification des éléments à nettoyer

```
# Observez le texte brut avant nettoyage
print(repr(roman[1000:1200]))   # repr() montre les caractères cachés (\
n, \t...)
```

3.2 Fonction de nettoyage

```
import re

def clean_text(text):
    """Nettoie un texte pour une analyse NLP en français."""
    # 1. Mise en minuscules
    text = text.lower()
    # 2. Suppression des marqueurs d'italique Gutenberg (_mot_)
    text = re.sub(r'_([^_]+)_', r'\1', text)
    # 3. Remplacement des sauts de ligne et tabulations par des espaces
    text = re.sub(r'[\n\r\t]+', ' ', text)
    # 4. Suppression de la ponctuation et des caractères spéciaux
    #    On conserve les lettres françaises accentuées
    text = re.sub(r"[^a-zàâçéèêëîïôûùüÿñæœ' ]", ' ', text)
    # 5. Normalisation des apostrophes typographiques
    text = re.sub(r"[''`]", "'", text)
    # 6. Suppression des espaces multiples
    text = re.sub(r'\s+', ' ', text)
    # 7. Suppression des espaces en début et fin
    text = text.strip()
    return text

roman_clean = clean_text(roman)

print(f'Taille avant : {len(roman):,} caractères')
print(f'Taille après : {len(roman_clean):,} caractères')
print(f'Réduction   : {(1 - len(roman_clean)/len(roman))*100:.1f}%')
print(f'\nAperçu nettoyé :\n{roman_clean[:400]}')
```

3.3 Vérification du nettoyage sur un extrait

```
extrait = roman[2800:3100]
print('AVANT NETTOYAGE :')
print(extrait)
print('\nAPRÈS NETTOYAGE :')
print(clean_text(extrait))
```

✏️ Travail demandé

a) Appliquez clean_text sur le roman. Affichez la taille avant et après.

b) Testez votre fonction sur cet extrait bruité :
'«Ah! c\'est vous, Dantès!» cria l\'homme... (numéro 34)\n\n-- Il est mort!!'
Que reste-t-il après nettoyage ?

c) Le nom 'Monte-Cristo' contient un tiret. Après nettoyage, devient-il 'monte cristo' (2 mots) ou 'montécristo' ? Proposez une solution pour le conserver en un seul token.

d) Modifiez clean_text pour également supprimer les nombres isolés (ex: '24', '1815').
Hint : re.sub(r'\b\d+\b', ' ', text)

e) Écrivez une version clean_text_leger qui ne fait que la mise en minuscule et la suppression des espaces multiples. Comparez les résultats.

Réponse :

a) Nettoyage du roman

Avec la fonction du sujet : **727 472 → 695 417 caractères (−4,4 %)**. La réduction est faible car on remplace la ponctuation par des espaces au lieu de la supprimer : l'essentiel du texte est conservé.

b) Extrait bruité

```
Avant : «Ah! c'est vous, Dantès!» cria l'homme... (numéro 34)\n\n-- Il est mort!!
Après : ah c'est vous dantès cria l'homme numéro il est mort
```

Il ne reste que des mots en minuscules : guillemets français, points d'exclamation, points de suspension, parenthèses, tirets de dialogue et retours à la ligne ont disparu. Le nombre « 34 » a lui aussi disparu, car la classe de caractères autorisés de l'étape 4 ne contient pas de chiffres. On perd ainsi « le numéro 34 », qui est pourtant l'identité de Dantès au château d'If (titre du chapitre XV).

c) « Monte-Cristo »

Le tiret est remplacé par un espace : on obtient **« monte cristo » (2 tokens)**, jamais « montécristo ». **Solution retenue (`clean_text_v2`)** : autoriser le tiret uniquement **entre deux lettres** puis supprimer les tirets isolés (`--` de dialogue). « monte-cristo » est alors conservé en un seul token (41 occurrences), tout comme « peut-être », « cent-jours » ou « saint-pierre-ès-liens ». Les inversions verbales sont séparées (« dit-il » → « dit il », « s'écria-t-il » → « s écria il ») pour ne pas créer de faux mots. Alternatives : remplacer les expressions connues par `monte_cristo` avant le nettoyage, ou utiliser `nltk.tokenize.MWETokenizer`.

d) Suppression des nombres + corrections apportées

Ajout de `re.sub(r'\b\d+\b', ' ', text)` (24, 1815…). Comme dit plus haut, cette étape est redondante avec la regex du sujet, mais elle devient nécessaire dès que l'on élargit la classe de caractères autorisés ; nous la gardons explicite. Nous avons aussi corrigé deux défauts de la fonction d'origine :

- l'ordre des étapes 4 et 5 : les apostrophes typographiques (') sont supprimées **avant** d'être normalisées, donc l'étape 5 ne sert à rien. On normalise désormais avant de filtrer ;
- les **élisions** : « l'abbé » et « abbé » étaient deux types distincts. On sépare l'élision (« l'abbé » → « l abbé »), en préservant « aujourd'hui » (pas de frontière de mot avant le d').

e) Version légère et comparaison

Version	Caractères	Mots (.split)	Types distincts
Texte brut	727 472	121 578	20 220
clean_text (sujet)	695 417	123 564	11 959
clean_text_v2 (retenue)	695 115	130 542	**11 142**
clean_text_leger	722 818	121 578	19 807

La version légère (minuscules + espaces) ne réduit presque pas le vocabulaire (19 807 types) : « dantès, », « dantès! », « _pharaon_ » restent distincts. Le nettoyage complet **divise le vocabulaire par 1,8**. Le nombre de mots augmente légèrement avec v2 (130 542) car les élisions et les inversions sont séparées en mots

autonomes.

❓ Questions de réflexion

Q1. Pourquoi faut-il nettoyer un texte avant de l'analyser en NLP ? Citez 3 raisons.

Q2. Quels problèmes peuvent apparaître si on n'effectue pas de nettoyage ? (au moins 3 exemples concrets sur ce roman)

Q3. Le nettoyage supprime-t-il de l'information utile ? Dans quel contexte la ponctuation pourrait-elle être informative ?

Q4. Faut-il toujours appliquer toutes les étapes de nettoyage ? Donnez un exemple où une étape serait contre-productive.

Réponse :

Q1. Pourquoi nettoyer ? Trois raisons

- **Réduire le bruit et la dimension** : de 20 220 à 11 142 types (−45 %), donc des matrices plus petites et moins creuses.
- **Uniformiser** des formes identiques : « Dantès » / « dantès » / « Dantès! » doivent être comptées ensemble.
- **Retirer les artefacts non linguistiques** : italiques Gutenberg `_Pharaon_`, tirets de dialogue `--`, table des matières, numérotation, mentions de licence.

Q2. Problèmes concrets sans nettoyage (sur ce roman)

- « Dantès », « Dantès! », « Dantès, », « «Dantès » comptés comme 4 mots différents ;
- « Pharaon » (nom du navire en italique) distinct de « Pharaon » ;
- les tirets de dialogue « -- » deviennent des tokens : 11 occurrences dans les seuls 5 000 premiers caractères ;
- des collages comme « Marseille.--L'arrivée » (titre du chapitre I) produisent un token absurde (SpaCy l'étiquette même comme adjectif) ;
- « l'abbé » et « abbé » séparés : sans traitement des élisions, l'abbé Faria est sous-compté.

Q3. Le nettoyage supprime-t-il de l'information utile ?

Oui. La **ponctuation** porte l'émotion et la modalité (« !! », « ? » : utile en analyse de sentiment), marque le dialogue (guillemets, tirets) et surtout délimite les phrases : nous l'avons constaté, `sent_tokenize` sur le texte nettoyé ne trouve **qu'une seule phrase** au lieu de 237. Les **majuscules** servent à la détection des noms propres (on a dû travailler sur le texte brut pour le WordCloud des PROPN). Les **nombres** portent les dates (24 février 1815) et les identités (numéros 34 et 27).

Q4. Faut-il toujours tout appliquer ?

Non, le nettoyage dépend de la tâche. Exemples contre-productifs : supprimer la ponctuation **avant** la segmentation en phrases (démontré en 4.3) ; passer en minuscules avant une **extraction d'entités nommées** (« Pharaon » le navire devient « pharaon ») ; supprimer les nombres pour une extraction de dates ; supprimer « ne… pas » en analyse de sentiment. Les modèles de type BERT/CamemBERT ont d'ailleurs leur propre tokeniseur et travaillent sur le texte quasi brut.

Partie 4

Tokenisation

La **tokenisation** est l'opération qui consiste à découper un texte en unités minimales d'analyse appelées **tokens**. Un token peut être un mot, un signe de ponctuation, un nombre ou même un sous-mot selon le niveau d'analyse choisi.

Type de token	Exemple extrait du roman	Usage
Mot entier	'dantès', 'château', 'vengeance'	Analyse fréquentiste, BoW, TF-IDF
Sous-mot (subword)	'dan##tès', 'château'	Modèles BERT, GPT
Caractère	'd','a','n','t','è','s'	Modèles de caractères, détection de fautes
N-gramme (bi-)	'château if', 'monte cristo'	Moteurs de recherche, expressions
Phrase	'Le 24 février 1815, la vigie…'	Résumé automatique, traduction

4.1 Tokenisation avec NLTK

```
from nltk.tokenize import word_tokenize

# Tokeniser le roman nettoyé
tokens = word_tokenize(roman_clean, language='french')

print(f'Nombre total de tokens : {len(tokens):,}')
print(f'Vocabulaire (tokens uniques) : {len(set(tokens)):,}')
print(f'Richesse lexicale : {len(set(tokens))/len(tokens):.4f}')
print(f'\n50 premiers tokens :')
print(tokens[:50])
```

4.2 Tokenisation avec SpaCy

```
# SpaCy — tokenisation plus précise pour le français
# Attention : SpaCy est lent sur un grand texte → on l'applique sur un
extrait
extrait_ch1 = roman_clean[:5000]

doc = nlp(extrait_ch1)
tokens_spacy = [token.text for token in doc if not token.is_space]

print(f'Tokens NLTK (extrait)  : {len(word_tokenize(extrait_ch1,
language="french"))}')
print(f'Tokens SpaCy (extrait) : {len(tokens_spacy)}')
print(f'\n30 premiers tokens SpaCy :')
print(tokens_spacy[:30])
```

4.3 Tokenisation en phrases

```
from nltk.tokenize import sent_tokenize

phrases = sent_tokenize(roman_clean[:20000], language='french')

print(f'Nombre de phrases (20 000 premiers caractères) : {len(phrases)}')
print(f'Longueur moyenne d\'une phrase : {sum(len(p.split()) for p in
phrases)/len(phrases):.1f} mots')
print(f'\nExemples de phrases :')
for p in phrases[:3]:
    print(f'  - {p[:100]}...')
```

4.4 N-grammes

```
from nltk import bigrams, trigrams

# Bigrammes (séquences de 2 mots consécutifs)
bi = list(bigrams(tokens[:200]))
print('Bigrammes (20 premiers) :')
print(bi[:20])

# Trigrammes
tri = list(trigrams(tokens[:200]))
print('\nTrigrammes (10 premiers) :')
print(tri[:10])
```

✏️ Travail demandé

a) Tokenisez le roman complet avec NLTK. Affichez les 50 premiers tokens.

b) Calculez et affichez :

- Le nombre total de tokens
- Le nombre de tokens uniques (vocabulaire)
- La richesse lexicale (ratio unique/total)

c) Comparez NLTK et SpaCy sur le premier chapitre (5000 premiers caractères).
Notez les différences dans un tableau.

d) Extrayez les 20 bigrammes les plus fréquents du roman.
Y a-t-il des noms de personnages ou des expressions caractéristiques ?
Hint : Counter(bigrams(tokens)).most_common(20)

e) Créez un DataFrame avec une ligne par token : colonnes [position, token, longueur].
Quelle est la longueur moyenne des tokens ?

Réponse :

a) et b) Tokenisation NLTK du roman complet

Tokens	Vocabulaire (types)	Richesse lexicale (TTR)	Temps
130 542	**11 142**	**0,0854**	0,5 s

```
['alexandre', 'dumas', 'le', 'comte', 'de', 'monte-cristo', 'tome', 'i', 'table',
'des', 'matières',
 'alexandre', 'dumas', 'i', 'marseille', 'l', 'arrivée', 'ii', 'le', 'père', 'et',
'le', 'fils', 'iii',
 'les', 'catalans', 'iv', 'complot', 'v', 'le', 'repas', 'des', 'fiançailles',
'vi', 'le', 'substitut',
 'du', 'procureur', 'du', 'roi', 'vii', 'l', 'interrogatoire', 'viii', 'le',
'château', 'd', 'if', 'ix', 'le']
```

Un TTR de 0,085 signifie qu'en moyenne chaque mot distinct est répété ≈ 12 fois : c'est normal pour un long texte (le TTR diminue mécaniquement avec la taille du corpus, il ne faut comparer que des textes de même longueur).

c) NLTK vs SpaCy sur le début du roman (5 000 caractères)

Critère	NLTK (word_tokenize)	SpaCy (fr_core_news_sm)
Tokens (texte nettoyé)	920	932
Tokens (texte brut)	975	1 039 (+6,6 %)
Élisions « l'abbé », « s'écria »	un seul token : `l'abbé`	deux tokens : `l'` + `abbé`
Inversion « est-il »	`est-il`	`est` + `-il`
Mots composés (monte-cristo, cent-jours)	conservés	découpés : `monte` - `cristo`
Tirets de dialogue « -- »	`--` (1 token)	`-` séparés ou collés au mot voisin
Italiques `_Pharaon_`	`_Pharaon_`	`_` + `Pharaon` + `_`
Notre-Dame-de-la-Garde, trois-mâts, M.	identiques	identiques
Informations supplémentaires	aucune	POS, lemme, stop word, entités…

d) 20 bigrammes les plus fréquents

Sur tous les tokens, ce sont des groupes grammaticaux : **de la** (589), **qu il** (575), **c est** (385), à la (384), de l (347), d un (253), à l (223), que je (222), d une (209), **l abbé** (201), il avait (195), que vous (186)… Seul « l abbé » renvoie au contenu. En filtrant les mots vides, des **noms de personnages et des expressions caractéristiques** apparaissent :

Bigramme	n	Bigramme	n	Bigramme	n
jeune homme	140	procureur roi	31	capitaine leclère	17
louis xviii	45	ministre police	22	thomson french	17
monsieur morrel	41	abbé faria	22	substitut procureur	16
jeune fille	40	edmond dantès	21	île monte-cristo	18
île elbe	36	monsieur villefort	21	écria dantès	19

« jeune homme » est la désignation récurrente de Dantès ; « louis xviii », « île d'Elbe », « ministre de la police » situent l'intrigue politique (Restauration / Cent-Jours) ; « procureur du roi » et « substitut » renvoient à Villefort ; « Thomson et French » est la maison de banque de Rome.

e) DataFrame des tokens

DataFrame de 130 542 lignes [position, token, longueur]. **Longueur moyenne : 4,32 caractères** (médiane 4, max 21 pour « saint-pierre-ès-liens »). Mais la longueur moyenne des **types** est de 7,60 caractères : les mots courts (de, le, la, il, à) sont peu nombreux mais extrêmement répétés, ce qui fait baisser la moyenne par occurrence.

❓ Questions de réflexion

Q1. Qu'est-ce qu'un token ? Donnez la différence entre un token et un mot.

Q2. Pourquoi la tokenisation est-elle une étape fondamentale en NLP ?

Q3. NLTK et SpaCy produisent-ils exactement les mêmes tokens ? Quelle est la principale différence ?

Q4. Dans quel cas l'analyse par bigrammes serait-elle plus utile que l'analyse par mots isolés ? Donnez un exemple concret.

Réponse :

Q1. Token vs mot

Un **token** est l'unité produite par un tokeniseur selon des règles techniques : mot, signe de ponctuation, nombre, élision, sous-mot. Un **mot** est une unité linguistique. Les deux ne coïncident pas toujours : « aujourd'hui » = 1 mot = 1 token ; « l'homme » = 2 mots, mais 1 token pour NLTK et 2 pour SpaCy ; « dit-il » = 2 mots ; pour BERT, « Dantès » peut devenir plusieurs sous-mots. On distingue aussi le token (occurrence, 130 542) du type (forme distincte, 11 142).

Q2. Pourquoi la tokenisation est-elle fondamentale ?

C'est elle qui définit les unités sur lesquelles tout le reste opère : comptages, stop words, stemming,

lemmatisation, vocabulaire et donc **les dimensions** des matrices BoW/TF-IDF. Une erreur à cette étape se propage dans tout le pipeline (ex. « l'abbé » non séparé → « abbé » sous-compté, Faria sous-estimé).

Q3. NLTK et SpaCy produisent-ils les mêmes tokens ?

Non (975 vs 1 039 tokens sur le même extrait brut). **Différence principale** : SpaCy possède des règles d'exception propres au français et **sépare les élisions** (l', d', s', qu') ainsi que les pronoms inversés (-il), alors que NLTK (Punkt + règles de type Treebank pensées pour l'anglais) les laisse collés. À l'inverse, NLTK garde les mots composés à tiret, que SpaCy découpe. SpaCy est aussi plus lent mais fournit POS, lemmes et entités.

Q4. Quand les bigrammes sont-ils plus utiles que les mots isolés ?

Pour les **expressions figées et les entités multi-mots** dont le sens n'est pas la somme des mots : « jeune homme » (140) ne veut pas dire « jeune » + « homme » séparément, « procureur du roi », « château d'If », « île d'Elbe », « Louis XVIII ». Aussi pour la **négation** (« pas coupable ») en analyse de sentiment, et dans un moteur de recherche (requête « abbé Faria » : mieux vaut chercher le bigramme que les deux mots indépendamment).

Partie 5

Analyse Fréquentielle

L'analyse de fréquence est la première approche statistique du texte. Elle permet d'identifier rapidement les mots dominants du corpus et de repérer la structure lexicale du document.

5.1 Fréquence brute des mots

```
from collections import Counter

# Compter la fréquence de chaque token
freq = Counter(tokens)

# Top 20 mots les plus fréquents
top20 = freq.most_common(20)

print('TOP 20 mots les plus fréquents :')
print(f'{"Rang":<6} {"Mot":<20} {"Fréquence":<10} {"% total"}')
print('-' * 50)
for rang, (mot, n) in enumerate(top20, 1):
    pct = n / len(tokens) * 100
    print(f'{rang:<6} {mot:<20} {n:<10} {pct:.3f}%')
```

5.2 Visualisation de la distribution

```
import matplotlib.pyplot as plt

mots = [m for m,n in top20]
freqs = [n for m,n in top20]

plt.figure(figsize=(12, 6))
bars = plt.barh(mots[::-1], freqs[::-1], color='steelblue',
edgecolor='white')
for bar, val in zip(bars, freqs[::-1]):
    plt.text(bar.get_width() + 50, bar.get_y() + bar.get_height()/2,
             str(val), va='center', fontsize=10)
plt.xlabel('Fréquence')
plt.title('Top 20 mots les plus fréquents dans Le Comte de Monte-Cristo')
plt.tight_layout()
plt.show()
```

5.3 Loi de Zipf

```
# Vérification empirique de la loi de Zipf
all_freqs = sorted(freq.values(), reverse=True)
rangs = range(1, len(all_freqs) + 1)

plt.figure(figsize=(10, 5))
plt.loglog(list(rangs), all_freqs, 'b.', markersize=2, alpha=0.5)
```

```
plt.xlabel('Rang (log)')
plt.ylabel('Fréquence (log)')
plt.title('Loi de Zipf – Le Comte de Monte-Cristo')
plt.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.show()

# Le mot le plus fréquent apparaît-il 2x plus que le 2e ?
top5 = freq.most_common(5)
print('Vérification de la loi de Zipf :')
for i, (m, n) in enumerate(top5):
    print(f'  Rang {i+1}: {m} ({n} fois) – ratio avec rang 1 : {top5[0]
[1]/n:.2f}')
```

✏️ Travail demandé

a) Calculez les fréquences et affichez le top 20 avec rang, mot, fréquence et pourcentage.
b) Tracez un graphique en barres horizontal du top 20.
c) Tracez le graphique log-log (loi de Zipf). Observez-vous une droite ?
La loi de Zipf est-elle vérifiée sur ce corpus ?
d) Quelle est la fréquence des mots 'dantès', 'vengeance', 'liberté', 'prison' ?
Calculez leur rang dans le classement général.
e) Combien de mots n'apparaissent qu'une seule fois (hapax legomena) ?
Quel pourcentage du vocabulaire représentent-ils ?
Ces mots rares sont-ils informatifs ou du bruit ?

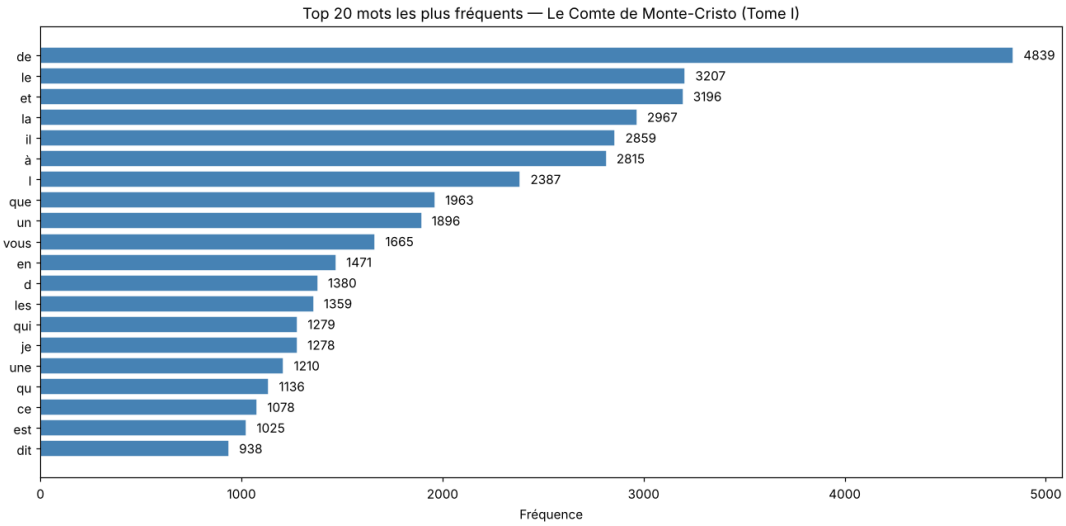
Réponse :

a) Top 20 (rang, mot, fréquence, %)

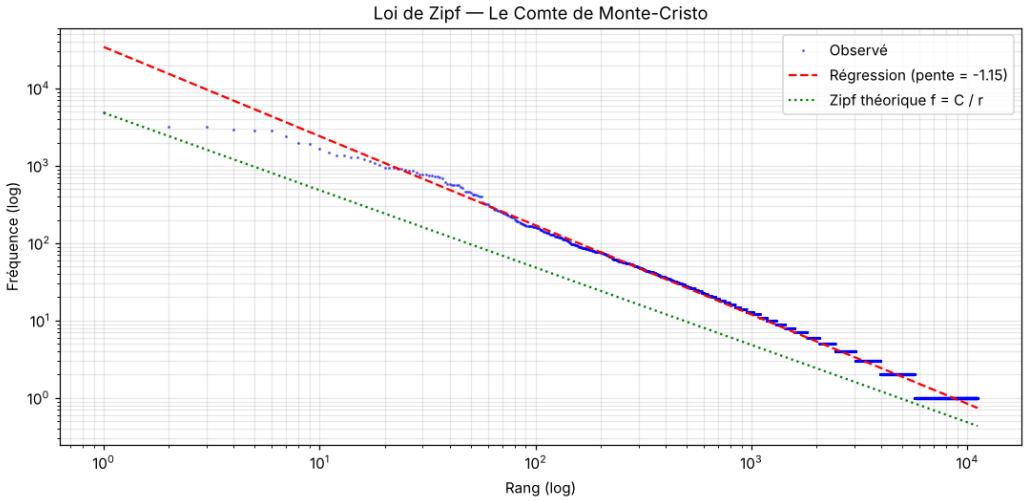
Rang	Mot	Fréq.	%	Rang	Mot	Fréq.	%
1	de	4839	3,707 %	11	en	1471	1,127 %
2	le	3207	2,457 %	12	d	1380	1,057 %
3	et	3196	2,448 %	13	les	1359	1,041 %
4	la	2967	2,273 %	14	qui	1279	0,980 %
5	il	2859	2,190 %	15	je	1278	0,979 %
6	à	2815	2,156 %	16	une	1210	0,927 %
7	l	2387	1,829 %	17	qu	1136	0,870 %
8	que	1963	1,504 %	18	ce	1078	0,826 %
9	un	1896	1,452 %	19	est	1025	0,785 %
10	vous	1665	1,275 %	20	dit	938	0,719 %

Ces 20 mots couvrent à eux seuls **30,6 % des tokens**.

b) Graphique du top 20



c) Loi de Zipf



Le nuage de points forme une **droite quasi parfaite** en log-log. La pente ajustée est **−1,15** (−1,09 sur les 1 000 premiers rangs), proche du −1 théorique : **la loi de Zipf est vérifiée**. Les écarts sont classiques : en tête, le ratio f_1/f_2 vaut 1,51 au lieu de 2 (les mots-outils les plus fréquents sont « tassés », ce que modélise mieux la loi de Zipf-Mandelbrot) ; en queue, des paliers horizontaux apparaissent (des milliers de mots à fréquence 1, 2, 3).

d) Mots thématiques

Mot	Fréquence	Rang
dantès	777	30
prison	67	220
liberté	49	301
vengeance	11	1 126

« vengeance » est étonnamment rare : le Tome I raconte l'ascension, la trahison, la prison et l'évasion de Dantès ; la vengeance proprement dite se déroule dans les tomes II à IV. La prison domine bien le vocabulaire thématique.

e) Hapax legomena

5 434 mots n'apparaissent qu'une fois, soit 48,8 % du vocabulaire mais seulement 4,16 % des tokens. Exemples : bêche, raviserait, langoureux, scindaient, indubitablement, exterminateur, licite, planton, cavités. Ils mélangent du **vocabulaire riche et informatif** (exterminateur, cavités), des **formes conjuguées rares** (raviserait, arriverai : la lemmatisation les ramènerait à des lemmes fréquents) et du **bruit**. Statistiquement, ils sont peu exploitables (un seul exemple) et doublent la dimension des matrices : on les filtre souvent (`min_df=2`), mais ils sont précieux en stylométrie et en recherche d'information.

❓ Questions de réflexion

Q1. Les 20 mots les plus fréquents sont-ils réellement informatifs sur le contenu du roman ? Pourquoi ?

Q2. Qu'est-ce que la loi de Zipf et quelle est sa signification pour le NLP ?

Q3. Qu'est-ce qu'un hapax legomena ? Doit-on les conserver pour une analyse statistique ?

Réponse :

Q1. Le top 20 est-il informatif ?

Non : ce sont tous des **mots-outils** (articles, prépositions, pronoms, auxiliaires). On obtiendrait presque la même liste avec n'importe quel roman français. Deux indices de style seulement : « vous » (10e) et « je » (15e) révèlent l'importance du dialogue, confirmée par « dit » (20e). Rien ne permet de deviner le sujet du roman.

Q2. La loi de Zipf et sa signification pour le NLP

Zipf énonce que la fréquence d'un mot est inversement proportionnelle à son rang : f(r) ≈ C / r^s avec s ≈ 1 (droite de pente −1 en log-log). Conséquences pour le NLP : quelques dizaines de mots couvrent un tiers du texte (→ intérêt des stop words), tandis qu'une très longue traîne de mots rares (→ hapax) rend les matrices **creuses**, impose de tronquer le vocabulaire (`max_features`, `min_df`), de lisser les probabilités des modèles de langue et de gérer les mots hors vocabulaire. C'est aussi la justification de la pondération IDF.

Q3. Hapax : définition, faut-il les garder ?

Un hapax est un mot qui n'apparaît **qu'une seule fois** dans le corpus (48,8 % du vocabulaire ici). Pour une analyse statistique (fréquences, BoW, classification, topic modeling) on les **supprime** en général : ils augmentent la dimension sans apporter d'information généralisable. On les **garde** pour la stylométrie (richesse lexicale d'un auteur), l'attribution d'auteur ou un moteur de recherche (un mot rare est très discriminant). Une lemmatisation préalable en élimine une partie.

Partie 6

Suppression des Stop Words

Les ***stop words*** (mots vides) sont des mots très fréquents dans une langue qui apportent peu d'information sémantique : articles (le, la, les), prépositions (de, à, dans), conjonctions (et, ou, mais), pronoms (il, elle, nous). Les supprimer réduit considérablement le bruit.

6.1 Chargement des stop words

```
from nltk.corpus import stopwords

sw_nltk = set(stopwords.words('french'))
sw_spacy = nlp.Defaults.stop_words

print(f'Stop words NLTK  : {len(sw_nltk)}')
print(f'Stop words SpaCy : {len(sw_spacy)}')

# Exemples
print(f'\nExemples NLTK  : {list(sw_nltk)[:15]}')
print(f'Exemples SpaCy : {list(sw_spacy)[:15]}')

# Dans NLTK mais pas dans SpaCy ?
diff = sw_nltk - sw_spacy
print(f'\nDans NLTK, absents de SpaCy : {list(diff)[:10]}')
```

6.2 Filtrage des tokens

```
# Stop words enrichis (spécifiques au roman)
sw_custom = sw_nltk | sw_spacy | {
    'dit', 'disant', 'répondit', 'reprit', 'ajouta', 'cria',
    'monsieur', 'madame', 'mademoiselle', 'car', 'alors', 'puis',
    'bien', 'encore', 'tout', 'très', 'plus', 'non', 'oui', 'si'
}

tokens_no_stop = [t for t in tokens
                  if t.lower() not in sw_custom
                  and len(t) > 2]

freq_clean = Counter(tokens_no_stop)

print(f'Tokens avant : {len(tokens):,}')
print(f'Tokens après : {len(tokens_no_stop):,}')
print(f'Réduction    : {(1-len(tokens_no_stop)/len(tokens))*100:.1f}%')

print('\nTop 20 mots après suppression des stop words :')
for mot, n in freq_clean.most_common(20):
    print(f'  {mot:<20} {n}')
```

6.3 Comparaison avant / après

```
# Comparer les top 10 avant et après suppression des stop words
print(f'{"Rang":<5} {"Avant":<22} {"Après"}')
print('-' * 55)
avant = [m for m,_ in freq.most_common(10)]
apres = [m for m,_ in freq_clean.most_common(10)]
for i, (a, b) in enumerate(zip(avant, apres), 1):
    print(f'{i:<5} {a:<22} {b}')
```

✏️ Travail demandé

a) Comparez les listes de stop words NLTK et SpaCy. Laquelle est la plus complète ?
b) Appliquez la suppression des stop words. Affichez le top 20 après filtrage.
Quels noms de personnages sont maintenant visibles ?
c) Calculez le taux de réduction du vocabulaire :
(len(tokens) - len(tokens_no_stop)) / len(tokens) * 100
d) Justifiez pourquoi ces mots ont été ajoutés aux stop words contextuels :
'dit', 'répondit', 'monsieur'. Sont-ils vraiment non-informatifs dans ce roman ?
e) Cherchez et comparez les fréquences de :
'dantès', 'danglars', 'fernand', 'villefort', 'faria', 'mercédès'
Quel personnage est le plus présent après suppression des stop words ?

Réponse :

a) NLTK vs SpaCy

NLTK	SpaCy	Communs	Union	Seulement NLTK	Seulement SpaCy
157	**507**	85	579	72	422

SpaCy est bien plus complète : elle ajoute des adverbes et connecteurs (ainsi, alors, afin, après, assez, autrement…), parfois en double sans accent (« anterieur »/« antérieur »). NLTK contient surtout les formes conjuguées d'être et avoir (eusse, aurions, ayants) et les lettres élidées (c, d, j, l…), absentes de SpaCy. Les deux listes sont complémentaires : nous utilisons leur union.

b) Top 20 après suppression

Mot	n	Mot	n	Mot	n	Mot	n
dantès	777	**caderousse**	219	fois	167	demanda	155
homme	424	jeune	217	**danglars**	166	yeux	154
morrel	273	**abbé**	214	rien	165	fit	141
villefort	258	père	193	**fernand**	161	point	139
edmond	247	faire	225	temps	157	jour	138

Les personnages apparaissent enfin : **Dantès/Edmond, Morrel, Villefort, Caderousse, l'abbé (Faria), Danglars, Fernand** (Mercédès est 21e). Restent quelques mots peu informatifs (faire, fois, point, fit, demanda).

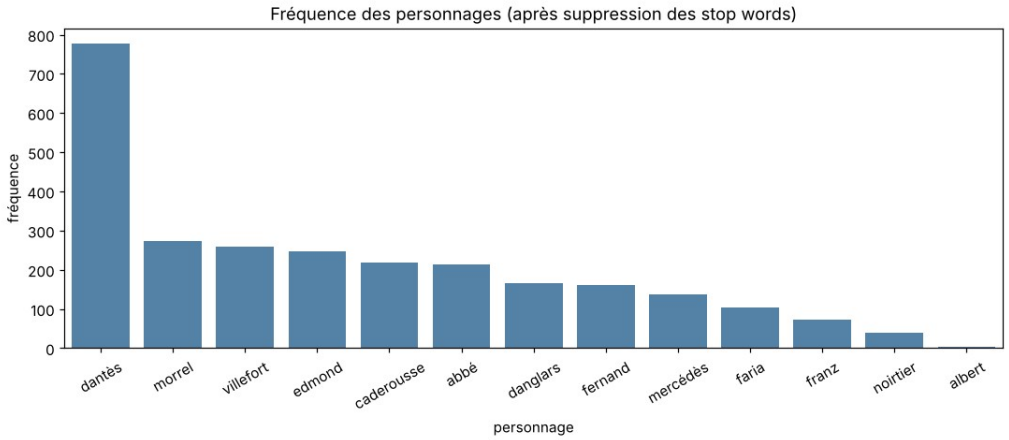
c) Taux de réduction

130 542 → 50 389 tokens : réduction de 61,4 %. En revanche, le vocabulaire ne baisse que de 4,3 % (11 142 → 10 668 types) : les stop words sont **peu nombreux mais extrêmement fréquents** (loi de Zipf).

d) Pourquoi « dit », « répondit », « monsieur » ?

« dit » apparaît 938 fois (20e mot du roman), « répondit » 137, « reprit » 119, « monsieur » 333. Ce sont les **incises de dialogue** (« dit Caderousse », « répondit Dantès » : après dit/répondit viennent surtout il, le, Caderousse, Dantès, Danglars) et une **formule de politesse** (« monsieur Morrel », « monsieur de Villefort »). Ils sont répartis uniformément et masquent les mots thématiques. Ils ne sont pourtant **pas totalement non-informatifs** : ils mesurent la part de dialogue, et « monsieur » traduit la hiérarchie sociale du XIXe siècle. La liste est aussi incohérente : « demanda » (155), « fit », « point » (« ne… point ») ou « faire » auraient dû y figurer.

e) Personnages



Dantès domine largement (777, et 1 024 avec « Edmond »), devant Morrel (273), Villefort (258), Caderousse (219), Danglars (166), Fernand (161), Mercédès (138) et Faria (105, mais 214 « abbé »).

❓ Questions de réflexion

Q1. Quels mots ont disparu après suppression des stop words ? Étaient-ils utiles ?

Q2. Pourquoi est-il parfois justifié d'ajouter des mots spécifiques au domaine dans la liste des stop words ?

Q3. Donnez un exemple de tâche NLP où il ne faudrait PAS supprimer les stop words.

Q4. La suppression des stop words est-elle réversible ? Quel impact a-t-elle sur la compréhension d'une phrase ?

Réponse :

Q1. Mots disparus

Articles (le, la, les, un), prépositions (de, à, en), conjonctions (et, que), pronoms (il, je, vous), auxiliaires (est, avait) et élisions (l, d, qu). Ils sont **inutiles pour identifier les thèmes**, mais indispensables à la syntaxe et au sens (négation, rôle des personnages, temps).

Q2. Pourquoi des stop words spécifiques au domaine ?

Certains mots sont fréquents **partout dans un corpus donné** sans discriminer les documents : « dit » et « monsieur » dans un roman dialogué du XIXe siècle, « patient » dans un corpus médical, « produit » dans des avis clients. Ils se comportent comme des mots vides dans ce contexte précis. C'est l'équivalent manuel de `max_df` (qui a d'ailleurs retiré automatiquement homme, jeune, yeux, faire, fois en partie 10).

Q3. Tâche où il ne faut PAS supprimer les stop words

L'analyse de sentiment : « je ne suis pas coupable » devient « coupable », sens inversé. Également la traduction automatique, les questions-réponses, les modèles de langue, la recherche d'expressions exactes (« le comte de Monte-Cristo ») et la **stylométrie / attribution d'auteur**, où la fréquence des mots-outils est justement le meilleur marqueur du style d'un auteur.

Q4. Est-ce réversible ? Impact sur la compréhension

Non, c'est une perte d'information irréversible (sauf à garder le texte original). Exemple : « Je ne l'ai pas trahi » → « trahi » : le sens est inversé. « Il est mort » → « mort ». On perd la négation, les relations sujet/objet, les temps : la phrase devient un sac de mots-clés, utile pour la recherche thématique mais plus pour la compréhension.

Partie 7

Visualisation — WordCloud

Un **nuage de mots** (WordCloud) est une représentation visuelle des mots d'un corpus où la taille de chaque mot est proportionnelle à sa fréquence. C'est un outil d'exploration rapide, utile pour identifier les thèmes dominants d'un texte.

7.1 WordCloud basique

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# Texte = tokens nettoyés réunis
texte_clean = ' '.join(tokens_no_stop)

wc = WordCloud(
    width           = 1200,
    height          = 600,
    background_color= 'white',
    colormap        = 'Blues',
    collocations    = False,   # Éviter les bigrammes
    max_words       = 150,
    min_font_size   = 8,
).generate(texte_clean)

plt.figure(figsize=(16, 8))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('WordCloud — Le Comte de Monte-Cristo', fontsize=16, pad=20)
plt.tight_layout()
plt.show()
```

7.2 WordCloud par personnage

```
personnages = ['dantès', 'danglars', 'fernand', 'villefort', 'mercédès']

fig, axes = plt.subplots(1, len(personnages), figsize=(20, 5))
colourmaps = ['Blues', 'Reds', 'Greens', 'Oranges', 'Purples']

for ax, perso, cmap in zip(axes, personnages, colourmaps):
    # Extraire les phrases contenant le personnage
    phrases_perso = [p for p in sent_tokenize(roman_clean,
language='french')
                     if perso in p]
    texte_perso = ' '.join(phrases_perso)
    tokens_perso = [t for t in word_tokenize(texte_perso,
language='french')
                    if t not in sw_custom and len(t) > 3]

    if tokens_perso:
        wc_perso = WordCloud(width=400, height=300,
background_color='white',
```

```
                              colormap=cmap, collocations=False).generate('
'.join(tokens_perso))
        ax.imshow(wc_perso, interpolation='bilinear')
    ax.axis('off')
    ax.set_title(perso.capitalize(), fontsize=12, fontweight='bold')

plt.suptitle('Champ lexical des personnages principaux', fontsize=14,
y=1.02)
plt.tight_layout()
plt.show()
```

✏️ Travail demandé

a) Générez un WordCloud du roman complet (après suppression des stop words).
Quels sont les 5 mots les plus visibles ? Sont-ils cohérents avec le roman ?

b) Générez 3 WordClouds pour les personnages : Dantès, Danglars et Villefort.
Quels sont les mots distinctifs de chacun ?

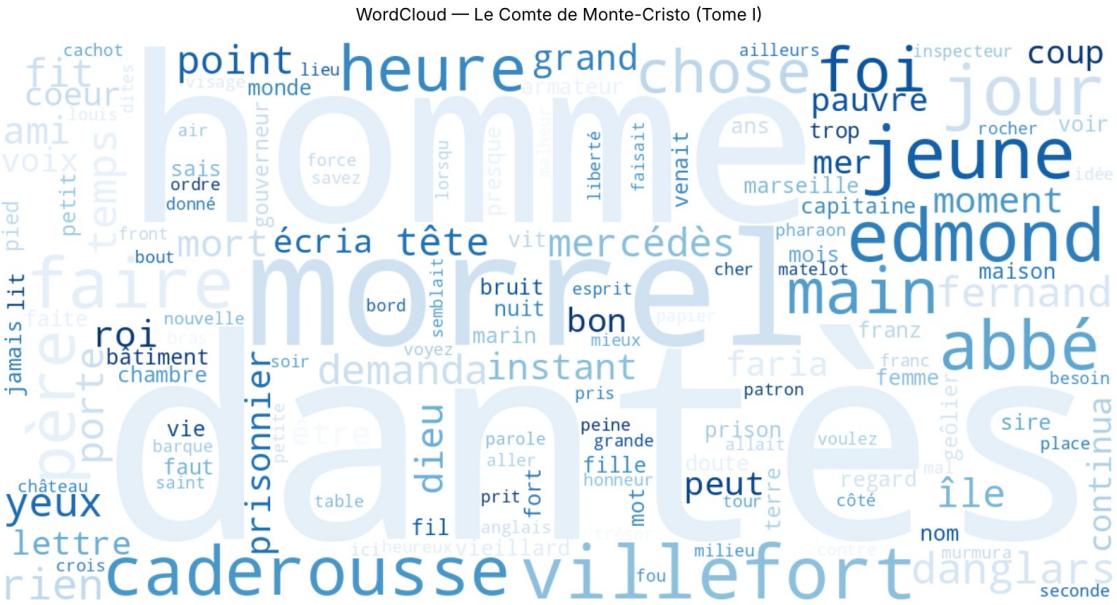
c) Créez un WordCloud uniquement avec les NOMS PROPRES (POS == PROPN avec SpaCy).
Quels lieux et personnages ressortent ?

d) Modifiez les paramètres du WordCloud :
- colormap='RdYlGn' (rouge → jaune → vert)
- max_words=50 vs max_words=200
Quel paramètre donne la visualisation la plus lisible ?

e) Limitez le WordCloud aux 3000 premiers mots (début du roman) vs les 3000 derniers.
Le vocabulaire est-il différent ? Qu'est-ce que cela révèle sur la structure narrative ?

Réponse :

a) WordCloud du roman complet

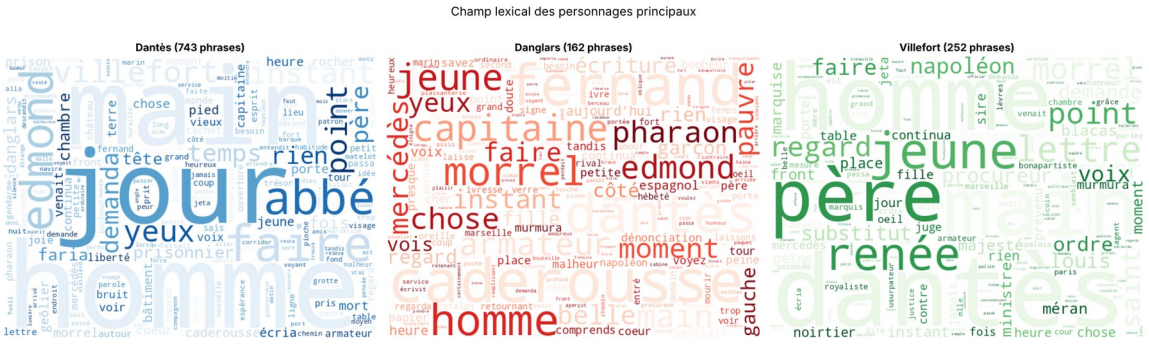


Les 5 mots les plus visibles : **dantès, homme, morrel, villefort, jeune**. C'est cohérent avec le Tome I : le héros, son armateur Morrel, le procureur Villefort qui l'emprisonne, et « jeune homme », désignation constante de Dantès. « homme » et « jeune » restent cependant génériques.

b) WordClouds par personnage

Correction du code fourni : `sent_tokenize(roman_clean)` ne trouve qu'**une seule phrase** car la ponctuation a été supprimée ; les trois nuages auraient été identiques (tout le roman). Nous segmentons

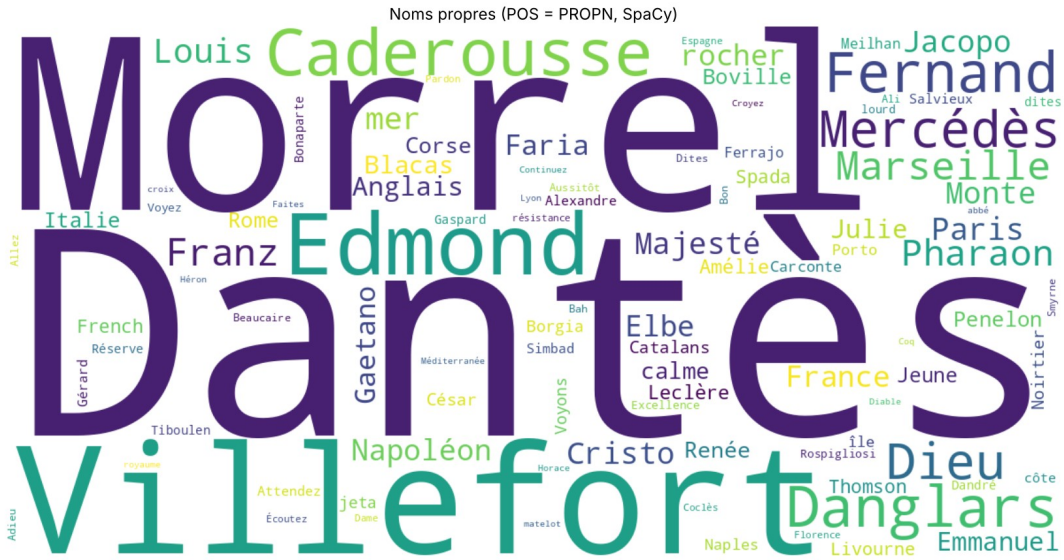
le texte **brut** en phrases (6 598), puis nettoyons chaque phrase. Le nom du personnage est retiré de son propre nuage, et nous calculons les **mots distinctifs** (sur-représentation par rapport au roman entier).



Personnage	Phrases	Mots distinctifs (sur-représentés)	Interprétation
Dantès	743	abbé, faria, prisonnier, fragments, dalles, pioche, crique	prison, tunnel, évasion, trésor
Danglars	162	écriture, dénonciation, fernand, caderousse, capitaine	le complot et la lettre anonyme
Villefort	252	certificat, prévenus, pétition, juge, saint-méran, renée, napoléon	justice, carrière, mariage, politique

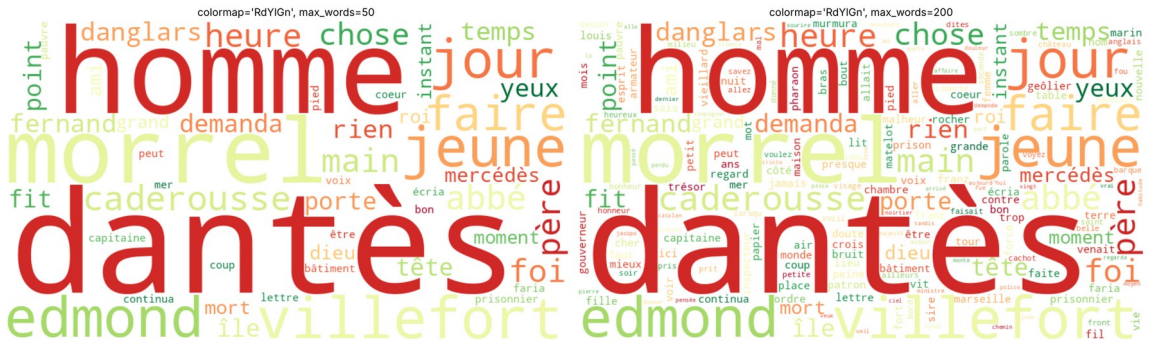
c) Noms propres (POS = PROPN)

La détection des noms propres s'appuie sur la majuscule : on applique donc SpaCy sur le texte **brut** (4 201 paragraphes, `nlp.pipe`, parser et NER désactivés : 6,7 s).



Personnages : Dantès (287), Morrel, Villefort, Edmond, Caderousse, Danglars, Fernand, Mercédès, Franz, Jacopo, Faria, Gaetano, Blacas, Julie, Emmanuel. **Lieux** : Marseille, Paris, France, île d'Elbe, Monte-Cristo. Également Dieu, Pharaon (le navire), Louis (XVIII), Napoléon. **Limites** : « Monte » et « Cristo » sont séparés, et des noms communs en début de phrase sont mal étiquetés (« mer », « rocher », « calme »).

d) Paramètres



max_words=50 est le plus lisible : peu de mots, grands, hiérarchie claire. Avec 200 mots, le nuage est plus exhaustif mais la majorité des mots devient illisible. La palette **RdYlGn** est peu adaptée : le jaune pâle sur fond blanc manque de contraste. Meilleur réglage : 50 à 100 mots avec une palette séquentielle sombre (Blues, viridis).

e) Début vs fin du roman



Les vocabulaires sont **très différents** (indice de Jaccard = 0,16). Début : dantès, père, capitaine, morrel, armateur, danglars, mercédès, fernand → Marseille, le navire, la famille, le triangle amoureux. Fin : franz, gaetano, barque, feu, hôte, excellence, simbad → l'Italie, Franz d'Épinay et le mystérieux « Simbad le marin ». Cela révèle la **structure narrative** : changement de décor, de personnages et de point de vue. Le héros a disparu du vocabulaire sous son propre nom, il est devenu « l'hôte », « Son Excellence », « Simbad ».

❓ Questions de réflexion

Q1. Les WordClouds sont-ils suffisants pour analyser un corpus ? Quelles sont leurs limites ?

Q2. Identifiez les thèmes dominants du roman à partir de votre WordCloud.

Q3. Pourquoi est-il important d'avoir supprimé les stop words AVANT de générer le WordCloud ?

Réponse :

Q1. Les WordClouds suffisent-ils ?

Non, ce sont des outils d'**exploration**. Limites : pas de valeur quantitative précise (tailles difficiles à comparer), les mots longs paraissent plus importants, aucune information de contexte, d'ordre ou de négation, forte dépendance au prétraitement, mise en page aléatoire, mots génériques fréquents surreprésentés (homme, faire), expressions multi-mots cassées (« monte » / « cristo »). Surtout, la fréquence brute ne dit pas ce qui est **distinctif** : il faut des mesures relatives (TF-IDF, log-ratio, comme fait en b).

Q2. Thèmes dominants

- **Les personnages et leurs relations** : Dantès, Morrel, Villefort, Danglars, Fernand, Mercédès, Caderousse ;
- **la mer et la navigation** : Pharaon, capitaine, navire, armateur, marin, barque, île ;
- **la justice et la prison** : procureur, substitut, prisonnier, cachot, geôlier, château (d'If), abbé ;
- **la politique** : roi, sire, Napoléon, Louis XVIII, île d'Elbe, ministre ;
- **la famille et l'amour** : père, mère, fils, fiançailles, Mercédès ; puis **le trésor et l'Italie** en fin de tome.

Q3. Pourquoi supprimer les stop words avant ?

La taille d'un mot étant proportionnelle à sa fréquence, « de, le, et, la, il » (30 % des tokens) occuperaient tout le nuage et écraseraient les mots de contenu, dans un nombre de places limité (`max_words`). Le nuage serait identique pour n'importe quel texte français et ne dirait rien du roman.

Partie 8

Stemming (Racinisation)

Le **stemming** (ou racinisation) consiste à réduire un mot à sa racine morphologique en appliquant des règles heuristiques de suppression de suffixes. Le résultat n'est pas toujours un mot valide, mais des formes différentes d'un même mot partagent généralement la même racine.

8.1 Snowball Stemmer (français)

```
from nltk.stem import SnowballStemmer

stemmer = SnowballStemmer('french')

# Exemples sur des mots du roman
mots_test = [
    'dantès', 'vengeance', 'venger', 'vengeur', 'vindicatif',
    'prison', 'prisonnier', 'emprisonner', 'emprisonnement',
    'liberté', 'libérer', 'libérateur', 'libération',
    'mer', 'marin', 'maritime', 'navire', 'naviguer',
]

print(f'{"Mot original":<22} {"Racine (stem)"}')
print('-' * 42)
for mot in mots_test:
    print(f'{mot:<22} {stemmer.stem(mot)}')
```

8.2 Application sur le corpus

```
# Stemming de tous les tokens nettoyés
tokens_stemmed = [stemmer.stem(t) for t in tokens_no_stop]

freq_stemmed = Counter(tokens_stemmed)

print(f'Vocabulaire avant stemming : {len(set(tokens_no_stop)):,} mots')
print(f'Vocabulaire après stemming : {len(set(tokens_stemmed)):,} racines')
print(f'Réduction du vocabulaire   : {(1 -
len(set(tokens_stemmed))/len(set(tokens_no_stop)))*100:.1f}%')

# Top 20 racines
print('\nTop 20 racines :')
for r, n in freq_stemmed.most_common(20):
    print(f'  {r:<20} {n}')
```

8.3 Familles de mots

```
from collections import defaultdict

# Regrouper les mots par racine commune
groupes = defaultdict(set)
```

```
for mot in set(tokens_no_stop):
    groupes[stemmer.stem(mot)].add(mot)

# Afficher les 10 familles les plus grandes
top_familles = sorted(groupes.items(), key=lambda x: len(x[1]),
reverse=True)[:10]
print('Familles de mots (10 plus grandes) :')
for racine, mots in top_familles:
    print(f'  [{racine:<15}] -> {sorted(mots)[:6]}')
```

✏️ Travail demandé

a) Appliquez le Snowball Stemmer sur les 4 familles de mots ci-dessus (vengeance, prison, liberté, mer). Toutes les formes partagent-elles la même racine ?

b) Calculez la réduction du vocabulaire grâce au stemming. Ce gain est-il significatif ?

c) Trouvez 3 exemples de 'faux amis du stemmer' : deux mots de sens différent qui ont la même racine après stemming.

d) Affichez le tableau suivant pour 15 mots caractéristiques du roman :

```
Mot original | Radical (Snowball) | Intelligible ?
-------------|--------------------|---------------
vengeance    |                    |
prisonnier   |                    |
...
```

e) Comparez les fréquences des racines 'veng-', 'pris-', 'liber-', 'dant-'. Quel thème est dominant dans le roman ?

Réponse :

a) Familles de mots

Famille	Mots → racines	Racines distinctes
vengeance	vengeance→vengeanc, venger→veng, vengeur→vengeur, vindicatif→vindiqu	4
prison	prison→prison, prisonnier→prisonni, emprisonner→emprison, emprisonnement→emprison	3
liberté	liberté→libert, libérer→liber, libérateur→liber, libération→liber	2
mer	mer→mer, marin→marin, maritime→maritim, navire→navir, naviguer→navigu	5

Non, aucune famille ne partage une racine unique. Le stemmer gère bien la flexion (prisonnier / prisonniers → prisonni, libérer / libération → liber) mais mal la **dérivation** (vengeance ≠ venger) ; et navire / mer / marin n'ont de toute façon pas la même étymologie : un stemmer ne capte pas la parenté sémantique.

b) Réduction du vocabulaire

10 668 mots → 5 581 racines : −47,7 %. Le gain est **très significatif** : la dimension est presque divisée par deux, surtout grâce à la conjugaison (familles de 20 formes : trouva, trouvait, trouvaient, trouver… → « trouv »). Top racines : dantes (777), homm (509), morrel, demand (266), villefort, edmond, **port (234)**, jeun, fair.

c) Faux amis du stemmer

Mots de sens différents	Racine commune
mère (parent) / **mer** (océan)	mer
porte (de la cellule) / **porter** (verbe) / **port** (de Marseille)	port
sort (destin) / **sortir** / **sortie**	sort

livre (ouvrage) / **livrer** (remettre)	livr
sommes (verbe être) / **somme** (montant, ou sommeil)	somm

Le cas « port » est grave pour ce roman : la 7e racine la plus fréquente (234) mélange le port de Marseille, la porte du cachot et le verbe porter.

d) 15 mots caractéristiques

Mot original	Radical (Snowball)	Intelligible ?	Mot original	Radical (Snowball)	Intelligible ?
vengeance	vengeanc	non	mariage	mariag	non
prisonnier	prisonni	non	procureur	procureur	oui
château	château	oui	emprisonné	emprison	non
liberté	libert	non	évasion	évas	non
navire	navir	non	fiançailles	fiançaill	non
trésor	trésor	oui	cachot	cachot	oui
abbé	abbé	oui	marseille	marseil	non
capitaine	capitain	non			

Seuls **5/15** radicaux sont des mots réels (ceux que le stemmer ne modifie pas) ; les 10 autres sont tronqués et illisibles (vengeanc, prisonni, fiançaill, marseil).

e) Fréquences des racines thématiques

Racine	Occurrences	Mots regroupés
dant-	780	dantès, dante
pris-	251	prison, prisonnier(s), prisons… mais aussi **pris, prise** (prendre !)
liber-	49	liberté
veng-	19	vengeance(s), venger, vengeur, vengent…

Après le personnage principal, **le thème de la prison domine** nettement le Tome I, loin devant la liberté et la vengeance. Mais la racine « pris- » est elle-même un faux ami (elle inclut le participe « pris » du verbe prendre) : le chiffre de 251 surestime le thème carcéral.

❓ Questions de réflexion

Q1. Quels sont les avantages du stemming par rapport à l'absence de normalisation ?
Q2. Quelles sont les limites du stemming ? Donnez 3 exemples concrets de résultats incorrects.
Q3. Pourquoi le Snowball Stemmer est-il préféré au Porter Stemmer pour le français ?
Q4. Dans quel contexte préféreriez-vous le stemming à la lemmatisation ?

Réponse :

Q1. Avantages du stemming

Il réduit fortement le vocabulaire (−47,7 %), regroupe les flexions (prisonnier/prisonniers, toutes les formes conjuguées), réduit la sparsité des matrices, améliore le **rappel** d'un moteur de recherche (« emprisonné » retrouve « emprisonnement »). Il est très rapide, ne nécessite ni dictionnaire ni modèle, et reste disponible pour de nombreuses langues.

Q2. Limites (exemples concrets)

- **Sur-racinisation** (sens différents fusionnés) : mère/mer → « mer », porte/port/porter → « port », pris/prison → « pris- » ;
- **Sous-racinisation** (même famille séparée) : venger → « veng » vs vengeance → « vengeanc » ; formes irrégulières non reconnues : fut (≠ être), allait (≠ aller), yeux (≠ œil), aperçut inchangé ;
- **Radicaux illisibles et noms propres abîmés** : prisonni, fiançaill, caderouss, marseil, et « dantès » → « dantes » (accent perdu).

Q3. Pourquoi Snowball plutôt que Porter pour le français ?

L'algorithme de Porter (1980) a été conçu **pour l'anglais** : ses règles visent -ing, -ed, -ation, -ness. Appliqué au français, il laisse les terminaisons françaises intactes ou coupe à tort. Snowball (« Porter 2 », conçu par Porter lui-même) est un cadre multilingue avec un **jeu de règles spécifique au français** : suffixes -ement, -

ité, -euse, -ation, terminaisons verbales (-aient, -erons, -issait), gestion des voyelles accentuées et des régions RV/R1/R2 propres à la morphologie française.

Q4. Quand préférer le stemming à la lemmatisation ?

Pour les **gros volumes ou le temps réel** (moteurs de recherche, indexation de millions de documents) où la vitesse compte (NLTK est ≈ 8 fois plus rapide ici) ; quand la lisibilité des termes importe peu (features BoW/TF-IDF pour un classifieur) ; quand on privilégie le rappel ; ou pour une langue sans bon lemmatiseur.

🧠 Partie 9

Lemmatisation avec SpaCy

La **lemmatisation** retourne la **forme canonique** (lemme) d'un mot en s'appuyant sur un dictionnaire morphologique et une analyse grammaticale. Contrairement au stemming, le résultat est toujours un mot valide de la langue.

> 🔍 **Différence clé : Stemming vs Lemmatisation**
>
> chevaux → Stemming : 'chev' | Lemmatisation : 'cheval'
> mangeait → Stemming : 'mang' | Lemmatisation : 'manger'
> meilleurs→ Stemming : 'meil' | Lemmatisation : 'bon'
> allaient → Stemming : 'all' | Lemmatisation : 'aller'
> ⚡ Stemming : rapide, heuristique, résultat parfois illisible
> ✨ Lemmatisation : précise, linguistiquement correcte, nécessite un dictionnaire

9.1 Lemmatisation avec SpaCy

```
# SpaCy est lent → on traite un extrait représentatif
extrait_analyse = roman[300:10000]

doc = nlp(extrait_analyse)

# Extraire les tokens avec leurs propriétés
rows = []
for token in doc:
    if not token.is_space:
        rows.append({
            'token'   : token.text,
            'lemme'   : token.lemma_,
            'POS'     : token.pos_,
            'est_stop': token.is_stop,
            'est_ponc': token.is_punct,
        })

df_tokens = pd.DataFrame(rows)
print(df_tokens.head(30).to_string(index=False))
```

9.2 Lemmes propres (sans stop words ni ponctuation)

```
lemmes_propres = [
    token.lemma_.lower()
    for token in doc
    if not token.is_stop
    and not token.is_punct
    and not token.is_space
    and len(token.text) > 2
]

freq_lemmes = Counter(lemmes_propres)
```

```
print('Top 20 lemmes :')
for lemme, n in freq_lemmes.most_common(20):
    print(f'  {lemme:<20} {n}')
```

9.3 Analyse grammaticale (POS tagging)

```
# Distribution des catégories grammaticales
pos_counts = Counter(token.pos_ for token in doc if not token.is_space)

pos_labels = {
    'NOUN' :'Nom commun',    'VERB' :'Verbe',
    'ADJ'  :'Adjectif',      'ADV'  :'Adverbe',
    'PROPN':'Nom propre',    'DET'  :'Déterminant',
    'PRON' :'Pronom',        'ADP'  :'Préposition',
    'PUNCT':'Ponctuation',   'CONJ' :'Conjonction',
}

print('Distribution POS (chapitre I) :')
for pos, n in pos_counts.most_common():
    label = pos_labels.get(pos, pos)
    pct = n / sum(pos_counts.values()) * 100
    print(f'  {label:<20} {n:>5}  ({pct:.1f}%)')
```

✏️ Travail demandé

a) Appliquez SpaCy sur le premier chapitre. Affichez le DataFrame complet.
b) Extrayez uniquement les NOMS (NOUN) lemmatisés. Top 15 noms ?
c) Extrayez les NOMS PROPRES (PROPN). Quels personnages et lieux ressortent ?
d) Tableau comparatif stemming vs lemmatisation sur 15 mots du roman :

Mot original	Stemming (Snowball)	Lemme (SpaCy)	Correct ?
navire			
prisonnier			
vengeance			
(+ 12 autres)			

e) Quelle méthode (stemming ou lemmatisation) est la plus adaptée à l'analyse littéraire ? Justifiez avec 3 exemples concrets du roman.

Réponse :

a) SpaCy sur le chapitre I complet

Chapitre I « Marseille. — L'arrivée » : **4 189 tokens analysés en 0,46 s**. Avant l'analyse, on retire les italiques `_Pharaon_` et on remplace les tirets de dialogue `--` (sinon SpaCy produit « Marseille.--L' » étiqueté ADJ). Le DataFrame complet (token, lemme, POS, est_stop, est_ponc) est exporté dans `outputs/chapitre1_spacy_tokens.csv` ; extrait :

token	lemme	POS	est_stop	est_ponc
Le	le	DET	True	False
vigie	vigie	NOUN	False	False
Notre-Dame	Notre-Dame	PROPN	False	False
signala	signala	ADJ ×	False	False

trois-mâts	trois-mâts	NOUN	False	False
venant	venir	VERB	False	False
Smyrne	Smyrne	PROPN	False	False
Trieste	trieste	NOUN ×	False	False

Erreurs visibles : « signala » (passé simple) pris pour un adjectif et non lemmatisé, « Trieste » pris pour un nom commun.

b) Top 15 noms communs lemmatisés

monsieur (32), **armateur (28)**, homme (19), **capitaine (16)**, dantè (14, erreur), **bâtiment (11)**, île (9), père (9), **navire (8)**, oeil (7), ordre (7), **pilote (6)**, **port (6)**, fils (6), paquet (6). Le lexique est maritime et commercial : c'est l'arrivée du Pharaon et le rapport de Dantès à son armateur. « dantè » révèle une erreur : Dantès est parfois étiqueté NOUN et le lemmatiseur retire le « s » final comme un pluriel.

c) Noms propres

Personnages : Morrel (21), Danglars (17), Dantès (15), Leclère (10, le capitaine mort en mer), Edmond (10), Mercédès (3). **Lieux** : Marseille, île d'Elbe et Porto-Ferrajo (escale du navire et lettre de Napoléon), Paris, Naples, Smyrne, If, Jaros, Pomègue (îles devant Marseille), la Réserve. **Navire** : Pharaon (8). Quelques erreurs : « jeta », « Pardon », « Excusez » étiquetés PROPN à cause de la majuscule après un tiret de dialogue.

d) Stemming vs lemmatisation (lemme pris dans le contexte réel du roman)

Mot original	Stemming (Snowball)	Lemme (SpaCy)	Stem correct ?	Lemme correct ?
navire	navir	navire	non	oui
prisonnier	prisonni	prisonnier	non	oui
vengeance	vengeanc	vengeance	non	oui
chevaux	cheval	cheval	oui	oui
yeux	yeux	yeux	non	non (œil)
allait	allait	aller	non	oui
fut	fut	être	non	oui
meilleurs	meilleur	meilleur	non	non (bon)
aperçut	aperçut	apercevoir	non	oui
gendarmes	gendarm	gendarme	non	oui
prisonniers	prisonni	prisonnier	non	oui
disait	dis	dire	non	oui
émotion	émot	émotion	non	oui
jeunes	jeun	jeune	non	oui
dantès	dantes	dantès	non	oui

Score : stemming 1/15, lemmatisation 13/15. Les deux échecs de SpaCy : « yeux » (étiqueté ADJ dans sa phrase, alors qu'ailleurs il donne bien « oeil ») et « meilleurs », laissé au comparatif.

e) Méthode la plus adaptée à l'analyse littéraire : la lemmatisation

- **fut → être** (stem « fut ») et **aperçut → apercevoir** (stem inchangé) : le récit est au passé simple, très irrégulier ; seul le lemmatiseur regroupe ces formes avec leur verbe.
- **disait → dire** (stem « dis », confondu avec « dis ») et **allait → aller** : verbes de parole et de mouvement, omniprésents dans le dialogue, correctement regroupés.
- **mère / mer** : le stemmer les fusionne (« mer »), le lemmatiseur garde deux lemmes distincts, indispensable dans un roman où la mer et la famille sont deux thèmes majeurs. Et les résultats restent lisibles (« prisonnier » et non « prisonni »), ce qui compte pour une interprétation littéraire.

❓ Questions de réflexion

Q1. Quelle est la différence fondamentale entre stemming et lemmatisation ?

Q2. La lemmatisation tient compte de la catégorie grammaticale (POS). En quoi est-ce important pour le mot 'vole' (voler vs vol) ?

Q3. Quelle méthode produit les meilleurs résultats sur un corpus littéraire du XIXe siècle ? Justifiez.

Q4. Quel est le coût computationnel de SpaCy vs NLTK ? Quand ce compromis est-il acceptable ?

Réponse :

Q1. Différence fondamentale

Le **stemming** tronque les suffixes par règles heuristiques, **sans contexte ni dictionnaire** : le résultat n'est pas forcément un mot (vengeanc). La **lemmatisation** s'appuie sur une analyse morphologique, un dictionnaire et la **catégorie grammaticale** pour renvoyer la forme canonique (infinitif, masculin singulier) : le résultat est toujours un mot de la langue (fut → être, chevaux → cheval).

Q2. Pourquoi le POS compte : l'exemple de « vole »

« vole » peut être un verbe (« l'oiseau vole », lemme **voler**) alors que le nom correspondant est « **vol** » (« un vol ») : sans POS, on ne sait pas s'il faut ramener la forme vers le verbe ou le nom. Les homographes sont nombreux dans le roman : « porte » (NOUN, la porte du cachot) vs « porte » (VERB, porter), « été » (la saison vs participe d'être), « sort » (le destin vs sortir), « livre ». Le stemmer, lui, les réduit tous à la même racine. Le POS ne résout cependant pas tout : « voler » (dérober) et « voler » (dans les airs) restent un seul lemme (c'est le rôle de la désambiguïsation lexicale).

Q3. Meilleure méthode pour un corpus littéraire du XIXe siècle

La **lemmatisation** (13/15 contre 1/15 ici) : le récit au passé simple et à l'imparfait (fut, aperçut, allait, disait) multiplie les formes irrégulières que le stemmer ne sait pas traiter. Une réserve : `fr_core_news_sm` est entraîné sur des textes contemporains (presse, web) ; il commet des erreurs sur les noms propres anciens (« dantè »), le passé simple (« signala » → ADJ) et les tournures littéraires. Un modèle plus grand (`fr_core_news_lg`, `fr_dep_news_trf`) ou Stanza améliorerait les résultats.

Q4. Coût computationnel SpaCy vs NLTK

Sur le chapitre I : NLTK (tokenisation + stemming) **56 ms** contre SpaCy (pipeline complet) **440 ms**, soit **≈ 8 fois plus lent**, plus le chargement du modèle et sa mémoire. Sur le roman entier, SpaCy prend 6,7 s même en désactivant parser et NER. Ce compromis est acceptable lorsque la **qualité linguistique** prime et que le corpus est de taille raisonnable ou traité hors ligne (analyse littéraire, extraction d'informations), en optimisant avec `nlp.pipe`, la désactivation des composants inutiles et le multiprocessing. Il ne l'est pas pour de l'indexation massive en temps réel.

Partie 10

Matrice Document-Terme & Bag of Words

La **matrice Document-Terme** est la représentation numérique fondamentale d'un corpus. Chaque ligne représente un document (ici, un chapitre), chaque colonne représente un mot du vocabulaire, et chaque cellule contient un compte ou un score.

10.1 Préparation des documents (chapitres)

```
import re

# Découper le roman en blocs de 2000 mots
mots = roman_clean.split()
taille_bloc = 2000
blocs = [' '.join(mots[i:i+taille_bloc])
         for i in range(0, min(len(mots), taille_bloc*20), taille_bloc)]

noms_blocs = [f'Bloc_{i+1:02d}' for i in range(len(blocs))]

print(f'Nombre de blocs : {len(blocs)}')
print(f'Taille du premier bloc : {len(blocs[0].split())} mots')
```

10.2 Construction de la matrice BoW

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(
    lowercase    = True,
    stop_words   = list(sw_custom),
    min_df       = 2,          # Mot présent dans au moins 2 blocs
    max_df       = 0.90,       # Mot présent dans moins de 90% des blocs
    max_features = 300,        # Garder les 300 mots les plus fréquents
)

X_bow = vectorizer.fit_transform(blocs)
vocab = vectorizer.get_feature_names_out()

bow_df = pd.DataFrame(
    X_bow.toarray(),
    index  = noms_blocs,
    columns= vocab
)

print(f'Matrice BoW : {X_bow.shape[0]} documents × {X_bow.shape[1]} mots')
print(bow_df.iloc[:5, :8])
```

10.3 Analyse de la matrice

```
# Quel mot est le plus fréquent globalement ?
```

```
print('Mots les plus fréquents (somme sur tous les blocs) :')
print(bow_df.sum().sort_values(ascending=False).head(10))

# Mots les plus fréquents dans le Bloc 1
print('\nTop mots dans Bloc_01 :')
print(bow_df.loc['Bloc_01'].sort_values(ascending=False).head(10))

# Densité de la matrice (% de cellules non nulles)
densite = (bow_df > 0).sum().sum() / (bow_df.shape[0] * bow_df.shape[1]) *
100
print(f'\nDensité de la matrice : {densite:.2f}%')
```

10.4 Heatmap de la matrice

```
# Sélectionner les 15 mots les plus fréquents pour la visualisation
top_mots = bow_df.sum().sort_values(ascending=False).head(15).index

plt.figure(figsize=(14, 8))
sns.heatmap(
    bow_df[top_mots],
    cmap='YlOrRd', linewidths=0.3, annot=False,
    xticklabels=True, yticklabels=True
)
plt.title('Matrice BoW — 15 mots les plus fréquents')
plt.xlabel('Mots')
plt.ylabel('Blocs du roman')
plt.tight_layout()
plt.show()
```

✏️ Travail demandé

a) Construisez la matrice BoW. Affichez ses dimensions (lignes × colonnes).
b) Que représente une ligne de la matrice ? Une colonne ? Une cellule ?
c) Quel bloc contient le plus de mots différents (vocabulaire le plus riche) ?
d) Trouvez les blocs où 'dantès' apparaît le plus. Puis 'danglars'. Puis 'faria'.
Ces blocs correspondent-ils aux parties du roman où ces personnages sont actifs ?
e) Tracez la heatmap. Commentez la distribution des mots au fil des blocs.
f) La matrice BoW est souvent creuse (sparse). Vérifiez : quel % de cellules est à 0 ?
Pourquoi cette sparsité est-elle un problème pour certains algorithmes ML ?

Réponse :

a) Dimensions

Matrice BoW : **20 documents × 300 mots**. Les 20 blocs de 2 000 mots couvrent 40 000 mots, soit **31 % du Tome I (chapitres I à XII)**. Remarque : `max_df=0.90` a retiré automatiquement homme, jeune, yeux, faire et fois (présents dans plus de 90 % des blocs) ; « dantès » (17/20 blocs = 85 %) est conservé.

b) Ligne, colonne, cellule

Une **ligne** = un document (un bloc de 2 000 mots) représenté par son vecteur de 300 compteurs. Une **colonne** = un terme du vocabulaire et sa distribution à travers les blocs. Une **cellule** = le nombre d'occurrences de ce terme dans ce bloc (ex. « armateur » apparaît 15 fois dans Bloc_01).

c) Bloc au vocabulaire le plus riche

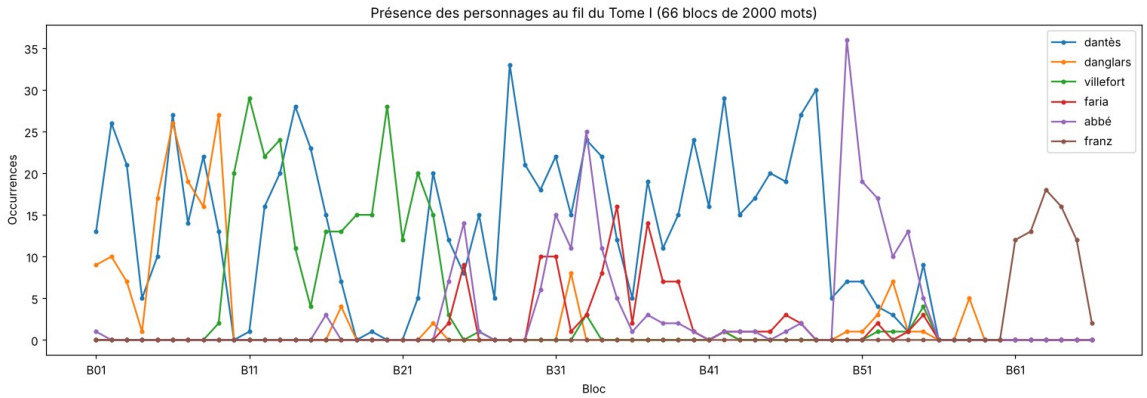
Bloc_17 (chapitres IX–X : fin du repas de fiançailles et cabinet des Tuileries) : **751 types distincts** (592 hors

stop words), et 151 des 300 mots du vocabulaire BoW non nuls. Il mélange deux scènes et deux registres (Marseille / la cour de Louis XVIII).

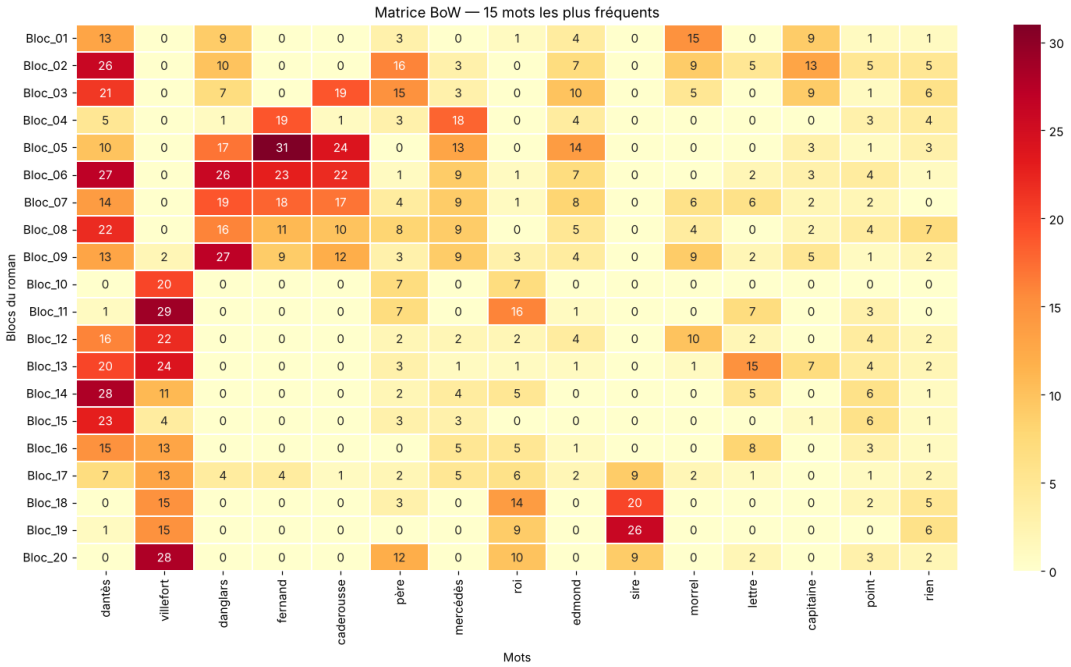
d) Où apparaissent Dantès, Danglars et Faria ?

Personnage	Bloc max	Top 3 blocs	Correspondance dans le roman
dantès	Bloc_14 (28)	14, 06, 02	ch. VII–VIII : interrogatoire et transfert au château d'If ; absent des blocs 10, 18, 20 (salons parisiens)
danglars	Bloc_09 (27)	09, 06, 07	ch. III–VI : le complot et l'arrestation au repas de fiançailles ; quasi absent ensuite (4 occurrences au bloc 17 seulement)
faria	— (0 partout)	—	Faria n'entre en scène qu'au ch. XV, hors des 20 premiers blocs

Pour Faria, nous avons étendu l'analyse au **roman entier (66 blocs)** : son pic est au bloc 35 (ch. XVIII, « Le trésor »). Les pics correspondent exactement aux parties où les personnages agissent. On voit même « abbé » culminer au ch. XXVI (l'abbé Busoni, c'est-à-dire Dantès déguisé) et « franz » au ch. XXXI, alors que « dantès » s'effondre.



e) Heatmap



La heatmap fait apparaître **deux grands actes** : blocs 1–9 (Marseille) dominés par danglars, fernand, caderousse, mercédès, père ; blocs 10–20 dominés par villefort, roi, sire (Paris, la politique, la justice). « dantès » est présent presque partout mais s'éteint dans les blocs 10, 18 et 20. Le BoW suffit déjà à retrouver la segmentation narrative.

f) Sparsité

Sur la matrice filtrée (20 × 300) : **54,9 % de zéros** (densité 45,1 %), une valeur faible car `max_features` ne garde que les mots fréquents. Sans filtrage, sur le roman entier : **66 × 11 142 avec 93,7 % de zéros** ; en

mémoire, 5,9 Mo en dense contre 0,56 Mo en format creux (CSR). La sparsité pose problème car elle augmente la mémoire et le calcul, rend les distances peu fiables (malédiction de la dimension : la plupart des documents ne partagent presque aucun terme), favorise le sur-apprentissage, et certains algorithmes exigent des données denses (KNN, PCA classique, réseaux de neurones). Solutions : formats creux, régularisation, réduction de dimension (SVD/LSA), embeddings denses.

❓ Questions de réflexion

Q1. Que représente une ligne de la matrice Document-Terme ? Et une colonne ?
Q2. Pourquoi la matrice BoW est-elle généralement très creuse (sparse) ?
Q3. Quels sont les avantages du Bag of Words ?
Q4. Quelles informations importantes sont perdues avec le modèle BoW ?

Réponse :

Q1. Ligne et colonne

Une **ligne** est la représentation vectorielle d'un document (son « profil lexical »). Une **colonne** est la distribution d'un terme sur l'ensemble des documents (où et combien il apparaît).

Q2. Pourquoi la matrice est-elle creuse ?

À cause de la **loi de Zipf** : chaque document n'utilise qu'une petite fraction du vocabulaire (≈ 750 types par bloc de 2 000 mots, contre 11 142 au total), et près de la moitié du vocabulaire est faite d'hapax qui n'apparaissent que dans un seul document.

Q3. Avantages du Bag of Words

Simple, rapide, sans apprentissage ; **interprétable** (chaque dimension est un mot) ; compatible avec tous les algorithmes de ML ; stockage efficace en format creux ; excellente base de référence en classification (Naive Bayes, régression logistique). Ici, il a suffi à retrouver la structure du roman et la présence des personnages.

Q4. Informations perdues

L'ordre des mots et la syntaxe : « Danglars dénonce Dantès » = « Dantès dénonce Danglars ». Le **contexte et la négation**, la **sémantique** (prison, cachot, geôle sont des dimensions orthogonales, sans aucune proximité), la polysémie (port/porte), la coréférence (« le jeune homme » = Dantès), et les mots hors vocabulaire. Les comptages dépendent aussi de la longueur des documents.



TF-IDF & Comparaison finale

Le **TF-IDF** (Term Frequency — Inverse Document Frequency) améliore le BoW en pondérant l'importance de chaque mot. Un mot fréquent dans un document mais rare dans l'ensemble du corpus reçoit un score élevé.

11.1 Formule TF-IDF

Composante	Formule	Signification
TF(t, d)	Fréquence(t,d) / Nb_mots(d)	Fréquence du terme t dans le document d
IDF(t)	log(N / (1 + df(t))) + 1	Rareté du terme t dans le corpus (N documents)
TF-IDF(t,d)	TF(t,d) × IDF(t)	Score final : élevé si rare dans corpus, fréquent dans doc

> 💡 **Intuition TF-IDF**
>
> Le mot 'le' (très fréquent dans tous les documents) → IDF faible → TF-IDF faible
>
> Le mot 'dantès' (fréquent dans certains blocs, rare ailleurs) → IDF élevé → TF-IDF élevé
>
> Le mot 'château' (fréquent dans les chapitres de prison uniquement) → TF-IDF très élevé dans ces blocs
>
> → TF-IDF discrimine mieux les documents entre eux que le simple comptage BoW

11.2 Calcul TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vec = TfidfVectorizer(
    lowercase    = True,
    stop_words   = list(sw_custom),
    min_df       = 2,
    max_df       = 0.90,
    max_features = 300,
    sublinear_tf = True,   # log(TF) pour réduire l'effet des mots très
fréquents
    use_idf      = True,
    smooth_idf   = True,
)

X_tfidf = tfidf_vec.fit_transform(blocs)
vocab_tfidf = tfidf_vec.get_feature_names_out()

tfidf_df = pd.DataFrame(
    X_tfidf.toarray().round(4),
    index  = noms_blocs,
    columns= vocab_tfidf
)
```

```
print(f'Matrice TF-IDF : {X_tfidf.shape}')
print(tfidf_df.iloc[:3, :6])
```

11.3 Mots signatures par bloc

```
print('Mots avec le TF-IDF le plus élevé par bloc :')
print(f'{"Bloc":<12} {"Top 6 mots (TF-IDF)"}')
print('-' * 65)
for i, bloc in enumerate(noms_blocs[:12]):
    top = tfidf_df.iloc[i].sort_values(ascending=False).head(6)
    print(f'{bloc:<12} {list(top.index)}')
```

11.4 Similarité cosinus entre blocs

```
from sklearn.metrics.pairwise import cosine_similarity

sim_matrix = cosine_similarity(X_tfidf)

sim_df = pd.DataFrame(
    sim_matrix.round(3),
    index  = noms_blocs,
    columns= noms_blocs
)

# Heatmap de similarité
plt.figure(figsize=(12, 10))
sns.heatmap(sim_df, cmap='Blues', annot=False,
            xticklabels=True, yticklabels=True, linewidths=0.2)
plt.title('Similarité cosinus entre les blocs (TF-IDF)')
plt.tight_layout()
plt.show()

# Blocs les plus similaires au Bloc 1
sim_bloc1 = sim_df['Bloc_01'].sort_values(ascending=False)[1:6]
print('Blocs les plus similaires au Bloc_01 :')
print(sim_bloc1)
```

11.5 Comparaison BoW vs TF-IDF

```
# Comparaison des scores pour le mot 'dantès' dans le Bloc_01
mot_test = 'dantès'

if mot_test in bow_df.columns:
    bow_score  = bow_df.loc['Bloc_01', mot_test]
    print(f'BoW  ({mot_test}) dans Bloc_01 : {bow_score}')

if mot_test in tfidf_df.columns:
    tfidf_score = tfidf_df.loc['Bloc_01', mot_test]
    print(f'TF-IDF ({mot_test}) dans Bloc_01 : {tfidf_score:.4f}')
```

```
# Comparaison des top mots par les deux méthodes
print('\nTop 10 par BoW   :',
list(bow_df.sum().sort_values(ascending=False).head(10).index))
print('Top 10 par TF-IDF:',
list(tfidf_df.sum().sort_values(ascending=False).head(10).index))
```

✏️ Travail demandé

a) Calculez la matrice TF-IDF et affichez les scores du Bloc_01.

b) Pour chaque bloc, affichez les 5 mots avec le TF-IDF le plus élevé.
Ces mots sont-ils pertinents pour résumer le contenu du bloc ?

c) Calculez la similarité cosinus entre tous les blocs.
Quels blocs sont les plus proches thématiquement ?

d) Utilisez TF-IDF pour un mini-moteur de recherche :
requete = 'château if prison dantès'
requete_vec = tfidf_vec.transform([requete])
scores = cosine_similarity(requete_vec, X_tfidf)[0]
blocs_trouves = sorted(zip(noms_blocs, scores), key=lambda x: -x[1])[:3]
print('Blocs les plus pertinents :', blocs_trouves)

e) Comparez les mots clés BoW vs TF-IDF pour le Bloc_01.
Quel résultat est le plus discriminant ?

Réponse :

a) Matrice TF-IDF et scores du Bloc_01

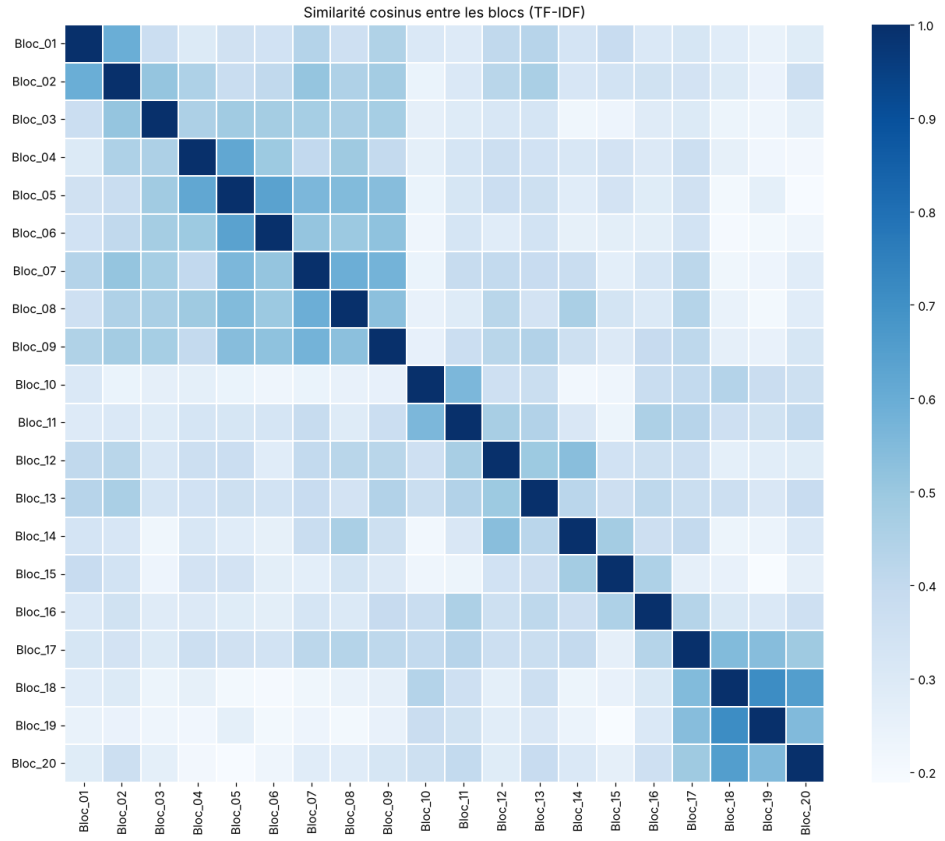
Matrice **20 × 300** (sublinear_tf, smooth_idf, normalisation L2). Meilleurs scores du Bloc_01 : armateur (0,198), navire (0,194), bâtiment (0,188), morrel (0,187), leclère (0,170), marin (0,168), poste (0,158), danglars, capitaine, île (0,152), port (0,148), château (0,148). Cela résume parfaitement le chapitre I : l'arrivée du Pharaon, la mort du capitaine Leclère, Dantès qui rend compte à l'armateur Morrel.

b) Top 5 TF-IDF par bloc

Bloc	Ch.	Top 5 TF-IDF	Bloc	Ch.	Top 5 TF-IDF
B01	I	armateur, navire, bâtiment, morrel, leclère	B11	VI–VII	renée, marquise, tuer, villefort, marquis
B02	I–II	armateur, leclère, capitaine, morrel, danglars	B12	VII	justice, villefort, politiques, morrel, substitut
B03	II	argent, vieillard, caderousse, capitaine, vin	B13	VII	villefort, ordres, lettre, juge, noirtier
B04	II–III	fernand, mère, larmes, attendez, mercédès	B14	VII–VIII	gendarmes, barque, palais, commissaire, lumière
B05	III	fernand, caderousse, catalan, danglars, malheur	B15	VIII	geôlier, gendarme, château, gouverneur, gendarmes
B06	III–IV	fernand, tuer, caderousse, danglars, papier	B16	VIII–IX	geôlier, gouverneur, marquis, impossible, marquise
B07	IV–V	fernand, caderousse, danglars, repas, viens	B17	IX–X	sire, cabinet, blacas, xviii, louis
B08	V	commissaire, fernand, danglars, caderousse, bruit	B18	X	sire, duc, blacas, majesté, baron
B09	V–VI	danglars, caderousse, fernand, armateur, morrel	B19	X–XI	sire, ministre, blacas, louis, xviii
B10	VI	marquise, napoléon, comte, villefort, saint-méran	B20	XI–XII	général, sire, ministre, villefort, noirtier

Oui, ces mots sont très pertinents : ils résument chaque passage. Misère du père Dantès (argent, vieillard, vin), les Catalans (fernand, mère, larmes, mercédès), le complot (tuer, papier), l'arrestation (commissaire), les fiançailles de Villefort (marquise, saint-méran, renée), l'interrogatoire et la lettre pour Noirtier (lettre, juge), le château d'If (geôlier, gouverneur), le cabinet de Louis XVIII (sire, blacas). Un peu de bruit subsiste (attendez, viens, impossible, bruit).

c) Similarité cosinus



Paires les plus proches : **Bloc_18–19 (0,709)** et 18–20 (0,653) : le cabinet des Tuileries ; 05–06 (0,637) et 04–05 (0,618) : les Catalans et le complot ; 01–02 (0,596) : l'arrivée du Pharaon. Les plus éloignées : 06–18 (0,203), 05–20, 15–19. Les blocs **consécutifs** sont les plus similaires (continuité des scènes) et la heatmap montre trois groupes : Marseille (1–9), Villefort et la prison (10–16), les Tuileries (17–20).

d) Mini moteur de recherche

Requête	Termes reconnus	Top 3 (score)	Pertinent ?
château if prison dantès	château, prison, dantès	B15 (0,296), B01, B14	oui : B15 = ch. VIII « Le château d'If »
navire capitaine mer	les 3	B01 (0,253), B13, B02	oui : arrivée du Pharaon
mariage fiançailles mercédès	fiançailles, mercédès	B06, B08, B07	oui : repas des fiançailles
complot lettre dénonciation	lettre seulement	B13, B11, B16	partiel : mots hors vocabulaire

Le moteur fonctionne. « if » est ignoré (moins de 3 lettres, hors vocabulaire), et « complot » et « dénonciation » ne sont pas dans les 300 mots retenus : c'est une limite de `max_features`.

e) BoW vs TF-IDF sur le Bloc_01

Rang	BoW	TF-IDF
1	armateur	armateur
2	morrel	navire
3	**dantès**	bâtiment
4	bâtiment	morrel
5	île	**leclère**
6	danglars	marin
7	navire	poste

TF-IDF est plus discriminant. « dantès » passe du 3e rang (BoW) au **23e** (IDF = 1,15, car présent dans 17 blocs sur 20) ; « leclère », propre à ce passage, monte du 13e au **5e** (IDF = 2,25). Mesuré globalement, le chevauchement moyen des top-5 entre deux blocs passe de **0,22 (BoW) à 0,09 (TF-IDF)** : les mots-clés TF-IDF caractérisent beaucoup mieux chaque bloc.

❓ Questions de réflexion

Q1. Pourquoi TF-IDF est-il souvent meilleur que BoW pour un moteur de recherche ?
Q2. Que signifie un score TF-IDF élevé pour un mot donné ?
Q3. Qu'est-ce que la similarité cosinus ? Quelle valeur indique une similarité totale ?
Q4. Dans quel cas TF-IDF pourrait-il donner de mauvais résultats ?

Réponse :

Q1. Pourquoi TF-IDF est meilleur pour un moteur de recherche

Parce qu'il pondère chaque terme par sa **rareté** : les mots présents partout (dantès, homme) pèsent peu, les mots spécifiques (geôlier, gouverneur) pèsent beaucoup. La requête est donc orientée vers le document où ses termes sont **caractéristiques**, et pas seulement fréquents. La normalisation L2 et le `sublinear_tf` limitent aussi l'effet de la longueur et des répétitions. Notre requête « château if prison dantès » trouve ainsi directement le chapitre du château d'If.

Q2. Que signifie un score TF-IDF élevé ?

Le mot est **fréquent dans ce document et rare dans le reste du corpus** : c'est un mot-signature du document (« geôlier » pour le Bloc_15, « leclère » pour le Bloc_01).

Q3. Similarité cosinus

C'est le cosinus de l'angle entre deux vecteurs : cos(A, B) = A·B / (‖A‖ × ‖B‖). Elle mesure l'orientation et non la longueur, donc elle est insensible à la taille des documents. Pour des vecteurs TF-IDF (positifs), elle varie de 0 (aucun terme commun) à **1 = similarité totale** (mêmes proportions de termes ; c'est la diagonale de la heatmap).

Q4. Quand TF-IDF donne-t-il de mauvais résultats ?

- **Synonymes et paraphrases** : « cachot », « prison », « geôle » n'ont aucun lien ; une requête « évasion » ne trouve pas un texte parlant de « fuite » ;
- **mots hors vocabulaire** (« dénonciation », « if » dans notre test) et textes très courts (requêtes, tweets) ;
- **termes importants mais omniprésents** : leur IDF devient faible (« dantès ») ou ils sont supprimés (max_df a retiré « homme ») ;
- **petit corpus** (20 documents) : IDF instable ; **polysémie** (port/porte) ; **ordre et négation** ignorés ; corpus multilingue.

Partie 12 — Tableau de synthèse

Comparaison complète des méthodes

Critère	Bag of Words (BoW)	TF-IDF	Votre évaluation
Simplicité	⭐⭐⭐ Très simple	⭐⭐ Simple	BoW plus simple, mais TF-IDF ne coûte qu'un paramètre de plus (même API scikit-learn, même vocabulaire).
Importance des mots	❌ Tous les mots égaux	✅ Mots rares valorisés	Confirmé : sur le Bloc_01, « dantès » passe du rang 3 (BoW) au rang 23 (TF-IDF), « leclère » du rang 13 au rang 5.
Prise en compte du contexte	❌ Aucune	❌ Aucune	Aucun pour les deux : « Danglars dénonce Dantès » = « Dantès dénonce Danglars ».
Performance classif.	⭐⭐ Correcte	⭐⭐⭐ Bonne	TF-IDF plus discriminant : chevauchement moyen des top-5 entre blocs 0,22 (BoW) contre 0,09 (TF-IDF).
Taille de la matrice	Grande (sparse)	Grande (sparse)	Identique (20 × 300). Sans filtrage : 66 × 11 142, avec 93,7 % de zéros.
Interprétabilité	✅ Facile	✅ Facile	Les deux sont lisibles ; le top TF-IDF résume chaque bloc (geôlier, gouverneur → château d'If).
Ordre des mots	❌ Ignoré	❌ Ignoré	Ignoré par les deux ; on peut le compenser en partie avec des n-grammes (ngram_range=(1,2)).
Robustesse aux hapax	❌ Sensible	✅ Moins sensible (IDF)	Nuance : l'IDF donne aux mots rares un poids maximal (risque de sur-valoriser un hapax) ; ici min_df=2 les élimine des deux.
Usage recommandé	Prototypage rapide	Moteur de recherche, classif.	BoW : base de référence et Naive Bayes. TF-IDF : notre moteur de recherche a trouvé le bon chapitre.

Alternative moderne	Word2Vec, BERT	Word2Vec, BERT, SBERT	Pour le français : CamemBERT / FlauBERT, et sentence-CamemBERT pour la recherche sémantique.

Bilan du pipeline complet

Étape	Opération	Ce qui change	Outil
1	**Chargement**	Texte brut → chaîne Python	`requests / open()`
2	**Nettoyage**	Suppression bruit, minuscules	`re (regex)`
3	**Tokenisation**	Texte → liste de tokens	`NLTK / SpaCy`
4	**Fréquences**	Tokens → comptage	`Counter`
5	**Stop words**	Tokens → tokens filtrés	`NLTK stopwords`
6	**WordCloud**	Tokens → visualisation	`WordCloud`
7	**Stemming**	Tokens → racines	`SnowballStemmer`
8	**Lemmatisation**	Tokens → lemmes grammaticaux	`SpaCy`
9	**BoW**	Textes → matrice numérique	`CountVectorizer`
10	**TF-IDF**	Textes → matrice pondérée	`TfidfVectorizer`

Questions de synthèse finale

Répondez à ces questions en vous appuyant sur l'ensemble du TP.

Q1
Pipeline NLP

Décrivez en vos propres mots les 10 étapes du pipeline NLP réalisé dans ce TP. Pour chaque étape, indiquez son objectif et ce qu'elle transforme (entrée → sortie).

Réponse :

Le pipeline transforme progressivement un fichier texte brut en vecteurs numériques exploitables par le machine learning :

#	Étape	Objectif	Entrée → Sortie
1	Chargement	Obtenir le texte de l'œuvre seule (sans licence)	URL / fichier → chaîne (727 472 car.)
2	Nettoyage	Supprimer le bruit, normaliser (minuscules, ponctuation, élisions, nombres)	chaîne → chaîne normalisée
3	Tokenisation	Découper en unités d'analyse (mots, phrases, n-grammes)	chaîne → liste de 130 542 tokens
4	Fréquences	Décrire statistiquement le corpus (top mots, Zipf, hapax)	tokens → Counter (mot → effectif)
5	Stop words	Retirer les mots-outils peu informatifs	130 542 → 50 389 tokens
6	WordCloud	Explorer visuellement les thèmes	tokens filtrés → image
7	Stemming	Regrouper les variantes par racine (rapide)	10 668 mots → 5 581 racines
8	Lemmatisation	Forme canonique selon le POS (précis)	tokens → lemmes + POS
9	BoW	Représenter chaque document par ses comptages	textes → matrice 20 × 300 d'entiers
10	TF-IDF	Pondérer par la rareté pour discriminer et comparer	textes → matrice 20 × 300 pondérée → similarités

Q2
Nettoyage vs Tokenisation

Expliquez la différence entre le nettoyage et la tokenisation. Peut-on tokeniser sans nettoyer ? Nettoyer sans tokeniser ? Justifiez.

Réponse :

Le **nettoyage** transforme le texte (chaîne → chaîne) : il supprime ou normalise des caractères (minuscules, ponctuation, nombres). La **tokenisation** segmente (chaîne → liste d'unités) sans modifier le contenu.

- **Tokeniser sans nettoyer : oui**, et c'est même parfois nécessaire. SpaCy et `sent_tokenize` sont conçus pour du texte brut (la segmentation en phrases et la NER ont besoin de la ponctuation et des majuscules), de même que les tokeniseurs de BERT. Sur le texte brut, on obtient cependant des tokens bruités (« -- », « Pharaon »).
- **Nettoyer sans tokeniser : oui en partie**, car les regex opèrent sur la chaîne. Mais une partie du nettoyage (stop words, stemming, filtre de longueur) se fait **au niveau du token** : on doit d'abord tokeniser.

L'ordre compte : nous avons montré que `sent_tokenize` après suppression de la ponctuation trouve 1 phrase au lieu de 237. En pratique, on segmente en phrases sur le brut, puis on nettoie, puis on tokenise en mots.

Q3
Stop words

Vous avez ajouté des mots comme 'dit', 'répondit', 'monsieur' aux stop words. Justifiez ce choix pour ce corpus. Ces mots seraient-ils à supprimer pour un corpus de tweets ou d'avis clients ?

Réponse :

Dans ce roman très dialogué du XIXe siècle, « dit » (938 occurrences, 20e mot du texte), « répondit » (137)

et « monsieur » (333) sont des **incises de dialogue et des formules de politesse** réparties uniformément : ils ne distinguent aucun passage et occupaient le haut des classements et des WordClouds. Les retirer fait apparaître les personnages et les thèmes.

Pour des tweets : non. « répondit » ou « monsieur » y sont rares ; les mots vides propres aux tweets sont plutôt « rt », « lol », « mdr », les URL, les @mentions. On y garderait les émojis et la négation, porteurs de sentiment. **Pour des avis clients : non plus**, et il faudrait être prudent avec les candidats évidents (« produit », « commande ») qui peuvent porter l'aspect évalué (« livraison » est essentiel en analyse par aspect). Une liste de stop words se construit **pour chaque corpus et chaque tâche**, de préférence à partir des données (fréquence documentaire, `max_df`) plutôt qu'à la main.

Q4
Stemming vs Lemmatisation

Comparez les deux méthodes de normalisation. Donnez 3 exemples concrets extraits du roman où la lemmatisation produit un meilleur résultat que le stemming.

Réponse :

Le **stemming** est rapide, sans dictionnaire, mais produit des racines illisibles et des confusions ; la **lemmatisation** est 8 fois plus lente mais linguistiquement correcte grâce au POS (13/15 corrects contre 1/15 sur notre tableau). Trois exemples du roman où la lemmatisation l'emporte :

- **« fut » → être** (stem : « fut ») et « aperçut » → apercevoir (stem inchangé) : le passé simple irrégulier, omniprésent dans le récit, n'est pas reconnu par le stemmer.
- **« disait » → dire** (stem : « dis », confondu avec « dis ») et « allait » → aller (stem : « allait ») : les verbes de parole et de mouvement sont correctement regroupés.
- **« mère » / « mer »** : fusionnés en « mer » par le stemmer, distincts pour le lemmatiseur ; de même « prisonniers » → « prisonnier » (lisible) au lieu de « prisonni ».

La lemmatisation n'est pas infaillible pour autant (« Dantès » → « dantè », « yeux » non ramené à « œil »).

Q5
BoW vs TF-IDF

Expliquez intuitivement pourquoi TF-IDF est supérieur à BoW pour un moteur de recherche. Utilisez un exemple concret avec des mots du roman.

Réponse :

Intuition : un moteur de recherche doit récompenser les mots qui **caractérisent** un document, pas ceux qui sont partout. Avec la requête « château if prison dantès » sur nos 20 blocs :

- en **BoW**, le Bloc_15 sort aussi en tête (0,551) mais l'écart est faible (Bloc_14 : 0,426), et les résultats suivants (Bloc_06, Bloc_02, Bloc_08 : les Catalans, l'arrivée, l'arrestation) ne remontent que parce que Dantès y est souvent nommé (27 et 26 fois) : « dantès », présent dans 17 blocs sur 20, pèse autant que « prison » ;
- en **TF-IDF**, « dantès » a un IDF faible (1,15) et ce sont « château » et « prison » qui comptent. Le **Bloc_15 (chapitre VIII « Le château d'If »)** l'emporte nettement (0,296, soit 1,9 fois le 2e résultat contre 1,3 en BoW), suivi de blocs qui mentionnent réellement le château d'If (Bloc_01 : « rasa le château d'If », Bloc_14 : le transfert).

C'est comme chercher un livre en bibliothèque : demander « un livre avec le mot “le” » est inutile, alors que « geôlier » mène directement au bon rayon. Même mécanisme pour « leclère » (rang 13 → 5 dans le Bloc_01).

Q6
Limites de ces approches

Citez 3 questions sur le roman auxquelles BoW et TF-IDF ne peuvent pas répondre (compréhension du sens, ironie, temporalité…). Quelle approche moderne (Word2Vec, BERT, GPT) permettrait de les aborder ?

Réponse :

BoW et TF-IDF ne voient que des **comptages de mots sans ordre ni sens**. Trois questions hors de leur portée :

- **« Qui est Simbad le marin ? Qui est l'abbé Busoni ? »** Il s'agit de Dantès sous d'autres identités. Il faut résoudre la **coréférence** et suivre une entité à travers ses noms : BoW compte « simbad », « abbé » et « dantès » comme des dimensions indépendantes.
- **« Danglars est-il sincère quand il félicite Dantès d'être nommé capitaine ? »** C'est une question d'**ironie** et d'intention cachée : les mots sont positifs, l'intention est hostile.
- **« La lettre de dénonciation est-elle la cause de l'arrestation, et qui l'a écrite, qui l'a postée ? »** Il s'agit de **temporalité, de causalité et de rôles** (Danglars écrit, Fernand poste) : « Danglars dénonce Dantès » et « Dantès dénonce Danglars » ont le même vecteur.

Approches modernes : **Word2Vec / FastText** capturent la similarité sémantique (prison ≈ cachot ≈ geôle) mais restent statiques (un vecteur par mot, sans contexte). **BERT (CamemBERT, FlauBERT pour le français)** produit des représentations **contextuelles** qui permettent la NER, la coréférence, la détection de sentiment ou d'ironie et les questions-réponses. **GPT et les LLM** vont plus loin : compréhension de longs contextes, raisonnement narratif, résumé et réponse en langage naturel à ces trois questions.

Annexes

A. Récapitulatif des fonctions Python

Fonction / Classe	Librairie	Rôle principal
requests.get(url)	requests	Télécharger un fichier depuis une URL
re.sub(pattern, repl, text)	re	Remplacer des motifs avec expressions régulières
word_tokenize(text, language=)	NLTK	Tokeniser en mots
sent_tokenize(text, language=)	NLTK	Tokeniser en phrases
stopwords.words('french')	NLTK	Liste des stop words français
SnowballStemmer('french')	NLTK	Racinisation en français
WordNetLemmatizer()	NLTK	Lemmatisation (anglais)
spacy.load('fr_core_news_sm')	SpaCy	Charger le modèle NLP français
token.lemma_	SpaCy	Lemme d'un token
token.pos_	SpaCy	Catégorie grammaticale (POS)
token.is_stop	SpaCy	True si stop word
CountVectorizer()	Scikit-learn	Matrice Bag of Words
TfidfVectorizer()	Scikit-learn	Matrice TF-IDF
cosine_similarity(A, B)	Scikit-learn	Similarité cosinus entre vecteurs
Counter(tokens)	collections	Comptage des fréquences
WordCloud().generate(text)	wordcloud	Générer un nuage de mots

B. Ressources pour aller plus loin

- NLTK Book (gratuit) : https://www.nltk.org/book/
- SpaCy — Documentation officielle : https://spacy.io/usage
- Scikit-learn — Text Feature Extraction : https://scikit-learn.org/stable/modules/feature_extraction.html
- Project Gutenberg (corpus littéraires libres) : https://www.gutenberg.org
- Python pour la Data Science (L. Galiana) : https://pythonds.linogaliana.fr/

🏆 **Pour aller plus loin — Perspectives**

Après ce TP fondamental, vous pouvez explorer :

- Word2Vec / GloVe : représentations vectorielles denses des mots (embeddings)
- CamemBERT / FlauBERT : modèles BERT adaptés au français
- Analyse de sentiment avec des lexiques ou des modèles supervisés
- Extraction d'entités nommées (NER) : personnages, lieux, dates
- Classification de textes : attribuer un chapitre à un auteur
- Clustering de documents : regrouper les chapitres par thème (K-Means + TF-IDF)

- LDA (Latent Dirichlet Allocation) : modélisation thématique non supervisée